

**Synligare utveckling av inbyggda fordonssystem –  
visualiserad kravhantering och samverkan**

**Visibler Development of Embedded Automotive Systems -  
Visualised Requirements Management and Collaboration**

|                            |                                       |
|----------------------------|---------------------------------------|
| <b>Report type</b>         | <b>Deliverable D2.1</b>               |
| <b>Report name</b>         | <b>Modelling Concepts and Methods</b> |
| <b>Dissemination level</b> | <b>Public</b>                         |
| <b>Status</b>              | <b>Initial Release</b>                |
| <b>Version number</b>      | <b>0.5</b>                            |
| <b>Date of preparation</b> | <b>2014-06-30</b>                     |

---

**Authors**

---

**Editor**

Urban Ingelsson

**E-mail**

urban.ingelsson@semcon.com

**Authors**

Henrik Lönn

**E-mail**[henrik.lonn@volvo.com](mailto:henrik.lonn@volvo.com)

Magnus Skoog

[magnus.skoog@autoliv.com](mailto:magnus.skoog@autoliv.com)**The Consortium**

---

Volvo Technology Corporation AB - Autoliv AB - Arccore AB- Systemite AB - Semcon Sweden AB

**Revision chart and history log**

---

| <b>Version</b> | <b>Date</b> | <b>Reason</b>                            |
|----------------|-------------|------------------------------------------|
| 0.1            | 2014-01-01  | Document skeleton                        |
| 0.2            | 2014-06-06  | Re-structuring                           |
| 0.3            | 2014-06-22  | Tightening of the material into a report |
| 0.4            | 2014-06-28  | New material added                       |
| 0.5            | 2014-06-30  | Initial Release                          |

## Table of contents

|                                                   |    |
|---------------------------------------------------|----|
| Authors.....                                      | 2  |
| Revision chart and history log.....               | 3  |
| Table of contents .....                           | 4  |
| 1 Terminology .....                               | 6  |
| 2 Introduction .....                              | 8  |
| 2.1 Purpose.....                                  | 9  |
| 2.2 Scope.....                                    | 9  |
| 2.2.1 View Categories.....                        | 9  |
| 2.2.2 Metric Categories.....                      | 10 |
| 2.2.3 Further scope.....                          | 10 |
| 2.3 Method .....                                  | 10 |
| 2.4 Document Outline .....                        | 11 |
| 3 Information Modelling.....                      | 12 |
| 3.1 Overview .....                                | 12 |
| 3.2 Manufacturer-Supplier Interaction.....        | 13 |
| 3.2.1 Traditional RFQ-Process .....               | 13 |
| 3.2.2 Suggested RFQ-Process .....                 | 14 |
| 3.2.3 Analysis of Requirements .....              | 15 |
| 3.2.4 Versioning in EAST-ADL.....                 | 15 |
| 3.2.5 Exchanging Models.....                      | 15 |
| 3.2.6 Integration Documentation .....             | 16 |
| 3.3 Task Model .....                              | 18 |
| 3.4 Tooling Support EAST-ADL models .....         | 20 |
| 3.5 Tooling Support Graphical Model Exchange..... | 22 |
| 3.5.1 Diagram Exchange Model.....                 | 23 |
| 4 Metrics .....                                   | 25 |
| 4.1 Requirement Progress .....                    | 25 |
| 4.2 Realization Progress.....                     | 28 |
| 4.3 Verification Progress.....                    | 28 |
| 4.4 Safety-related Progress Metrics.....          | 29 |
| 4.5 Product Metrics .....                         | 30 |
| 4.6 Identifying Context. ....                     | 31 |
| 5 Views.....                                      | 32 |
| 5.1 Feature-based Architecture Views .....        | 32 |
| 5.1.1 Example View .....                          | 32 |

|       |                                                                  |    |
|-------|------------------------------------------------------------------|----|
| 5.1.2 | <i>Related Modeling Concepts</i> .....                           | 32 |
| 5.2   | Requirements vs. Architecture .....                              | 33 |
| 5.2.1 | <i>Example View</i> .....                                        | 34 |
| 5.2.2 | <i>Modelling Concepts</i> .....                                  | 34 |
| 5.3   | Software Components, Ports and Signals .....                     | 34 |
| 5.3.1 | <i>Example View</i> .....                                        | 34 |
| 5.3.2 | <i>Relation to EAST-ADL</i> .....                                | 36 |
| 5.5   | Item Definition .....                                            | 37 |
| 5.5.1 | <i>Example View</i> .....                                        | 37 |
| 5.5.2 | <i>Related Modeling Concepts</i> .....                           | 37 |
| 5.6   | Functional and Technical Safety Concept .....                    | 39 |
| 5.6.1 | <i>Example View</i> .....                                        | 39 |
| 5.6.2 | <i>Modelling Concepts</i> .....                                  | 40 |
| 5.7   | Error Propagation Modelling .....                                | 40 |
| 5.7.1 | <i>Example View</i> .....                                        | 40 |
| 5.7.2 | <i>Related Modeling Concepts</i> .....                           | 41 |
| 5.7.3 | <i>Work-Phases when Eliciting Error Propagation Models</i> ..... | 43 |
| 6     | Summary .....                                                    | 45 |
| 6.1   | Inputs to WP3 .....                                              | 45 |
| 6.2   | Dissemination .....                                              | 45 |
| 7     | Bibliography .....                                               | 46 |

## 1 Terminology

Below is a set of terms used in project Synligare. The list largely excludes metamodel elements from AUTOSAR and EAST-ADL, as these are defined in the respective specification. Most of this terminology was defined in Synligare Deliverable D1.1 [9].

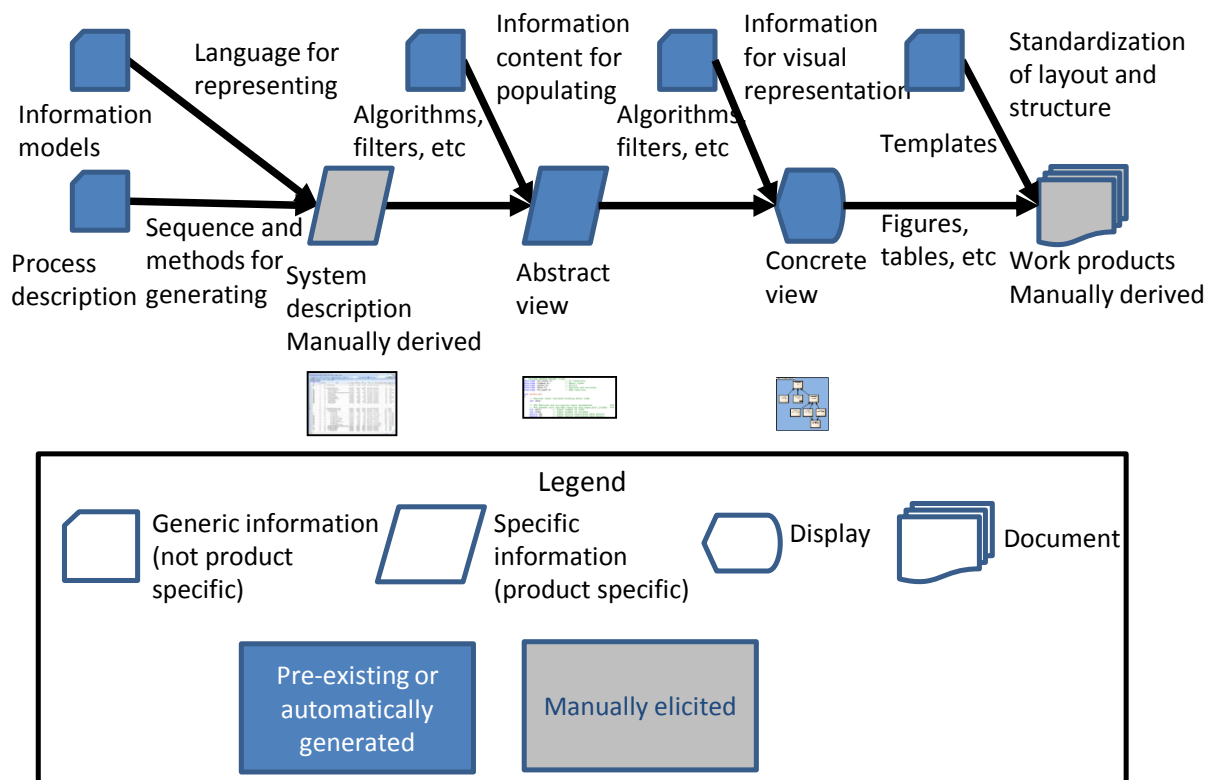
| Term                   | Description                                                                                                                                                       |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Abstract view          | A subset of a system description                                                                                                                                  |
| Artifact               | A general term for engineering elements that compose a solution                                                                                                   |
| Component              | A general term for software, hardware and functional components                                                                                                   |
| Concrete view          | The information of an abstract view, as it is shown to a software tool user. For example, it can have layout, coloring, etc.                                      |
| DIA                    | Development Interface Agreement                                                                                                                                   |
| Element                | A general term for parts of a model.                                                                                                                              |
| Embedded system        | Software and electronics embedded in a product                                                                                                                    |
| FROP                   | Feature Roll Out Plan                                                                                                                                             |
| Function Point         | A measurement of business functionality                                                                                                                           |
| Legacy tools           | Tools that already exist in a specific context                                                                                                                    |
| Metric                 | A quantification of aspect(s) of a design, system, project, organization , etc.                                                                                   |
| OEM                    | Original Equipment Manufacturer                                                                                                                                   |
| Realize                | Relationship between requirement and system model element                                                                                                         |
| Requirement            | A singular documented physical and functional need that a particular design, product or process must be able to perform                                           |
| Requirement management | The process of documenting, analyzing, tracing, prioritizing and agreeing on requirements and then controlling change and communicating to relevant stakeholders. |
| RFI                    | Request for Information                                                                                                                                           |
| RFQ                    | Request for Quotation                                                                                                                                             |
| Satisfy                | Relationship between requirements                                                                                                                                 |
| SEooC                  | Safety Element out of Context                                                                                                                                     |
| SIL                    | Safety Integrity Level                                                                                                                                            |
| Synligare              | FFI Project Grant FFI 2013-01296                                                                                                                                  |
| System Model           | A set of elements and relations that represents a system                                                                                                          |
| TRL                    | Technology Readiness Level, a measure introduced by NASA to characterize technology maturity from 1 (initial concept) to 9 (deployed technology)                  |
| Tier-n Supplier        | A supplier that provides components directly to an OEM (Tier-1) or Tier-i supplier (tier-i+1)                                                                     |
| User Stories           | Short description of functionality in informal language                                                                                                           |
| View                   | A representation of a whole system (or aspects thereof) from the perspective of a related set of concerns                                                         |
| Viewpoint              | A specification of the conventions for constructing and using a view.                                                                                             |
| WP                     | Work Package, one of five parts of Project Synligare, e.g. WP2                                                                                                    |

|              |                                                                            |
|--------------|----------------------------------------------------------------------------|
| Work product | The tangible result of a particular methodology or process step            |
| Well-formed  | The property of a model that it complies with applicable syntactical rules |
| XML          | Extensible Markup Language                                                 |

## 2 Introduction

This document is the report on results of Work Package 2 (WP2) of Project Synligare.

In Synligare Deliverable D1.1 [9], the increasing complexity of specifications for automotive embedded systems was identified as a problem that Project Synligare is to address. The key idea behind Project Synligare is to provide overview and to ease management of system specification by means of defining structured information models for representing system specifications. The structured information models will enable us to define views and metrics as well as algorithms for automatically generating them. The views and metrics will provide the overview of the system specification and system model to the engineers developing the embedded system.



**Figure 1. How information models relate to system description and views**

It can be seen in Figure 1 how information models, process descriptions, templates and algorithms for generating graphs are not specific to the product. These types of information can be used by an organization to standardize how they conduct their system development projects to ensure compatibility with various tools and quality systems, to gain various benefits. For example, it can be beneficial to purposefully structure information to be easily communicated into a supplier's or a customer's system. **Information models** describe how the system model should be represented, i.e. what components, relationships and attributes there should be. **Process descriptions** describe in what order and by what method to generate the system model as well as criteria for passing milestones in the development project. An **abstract view** is a subset of the system model selected to support a particular activity. A **concrete view** contains the same information as the abstract view, but is represented to a stakeholder to give a user-friendly overview over the information. Similarly, an **abstract metric** is the information required to calculate a metric. A **concrete metric** is the calculated value for the metric. It is expected that **algorithms** or rule sets are required to generate (or draw) the concrete view from the abstract view. However, these algorithms can be generic (they do not need to be system-specific), as long as the abstract view (and the rest of the system model) is represented in accordance with the information model. Often



the concrete view is a set of graphs, tables and other illustrations. With the help of **templates**, these graphs and tables can be used in **work products** such as the formal documentation of the system under development. It should be noted that describing the system, i.e. eliciting the system description, is a manual process. When information has been entered into the system description as specified in the information models, generation of concrete views can be automated by algorithms that assume information structured according to the information models. The assembly of figures and tables into work products is also a manual process, but is a good example of an activity that is made easier by automatic generation of concrete views.

Defining structured information models involves determining what information carriers (components, relationships and attributes) are required in the various design and verification activities. It should be considered what algorithms are needed to be carried out, like searches and sorting, to arrive at an information model that serves well to represent system specifications. A starting point is to consider the existing specification languages like EAST-ADL. Further, it should be considered how to do management of specifications and requirements in accordance with ISO 26262. The aim is that the outcomes of Project Synligare should be tried on an example system, to validate the structured information models, the views and the metrics.

This chapter introduces WP2 by defining purpose, scope and method.

---

## 2.1 Purpose

---

The work in WP2 will define structured information models that enable communication of system models in collaborative development of automotive embedded systems with the help of views and metrics. In other words, WP2 will develop a representation for system models for communication between OEM and Tier-1 (marked as “information model” in Figure 1), as well as conceptual solutions for visualizing information from the system model (marked as “algorithms” in Figure 1). The conceptual solutions are to be handed on to WP3, for development of corresponding prototype tools. With the help of those prototype tools, the conceptual solutions will be validated with respect to an example system in WP4. The expected effect of having appropriate structured information models for specification and appropriate views and metrics is increased predictability of safety, quality and performance of the specified system. Further it is expected that with a common understanding of how the specification is represented, visualized and measured, communication of specification should involve less misunderstanding. The collaboration aspect shall be considered such that a specification according to the structured information models can be modularized and communicated appropriately.

---

## 2.2 Scope

---

The scope of WP2 is defined by the following categories of views and metrics (that were identified WP1 and reported in D1.1 (1)).

---

### 2.2.1 View Categories

---

- Requirements
  - Clustered on Features/functions/components
  - Showing relevant attributes
  - Multiple aspects according to selected order and sorting
- Safety
  - All safety goals for a certain item
  - Functional Safety Concept
  - Technical Safety Concept
- System and Software Architecture

- All elements realizing a certain Feature
  - Function allocation to software architecture
- Progress
  - Requirements coverage
  - Element coverage (feature, function, ...)

Examples of views from these categories can be found in Chapter 5, albeit not in this order.

---

### 2.2.2 Metric Categories

---

- Dependability
- Number of completed analyses, completed artefacts, number of reviews
- Requirements validation, verification progress
- Impact of monitored system on production volume, income and cost

Examples of metrics from these categories can be found in Chapter 4, albeit not in this order.

---

### 2.2.3 Further scope

---

The view categories and metric categories listed above are addressed to ensure that the structured information models that are to be defined can support generation of those views and metrics. Information modelling results are presented in Chapter 3 of this document.

Given the focus on OEM-Tier1 interaction, an RFQ-process is described in Chapter 3 of this document. The views and metrics that are relevant for the RFQ-process are prioritized to focus on finding solutions that support collaborative system development.

The scope includes ensuring that the structured information models are adequate to specify an automotive embedded system.

---

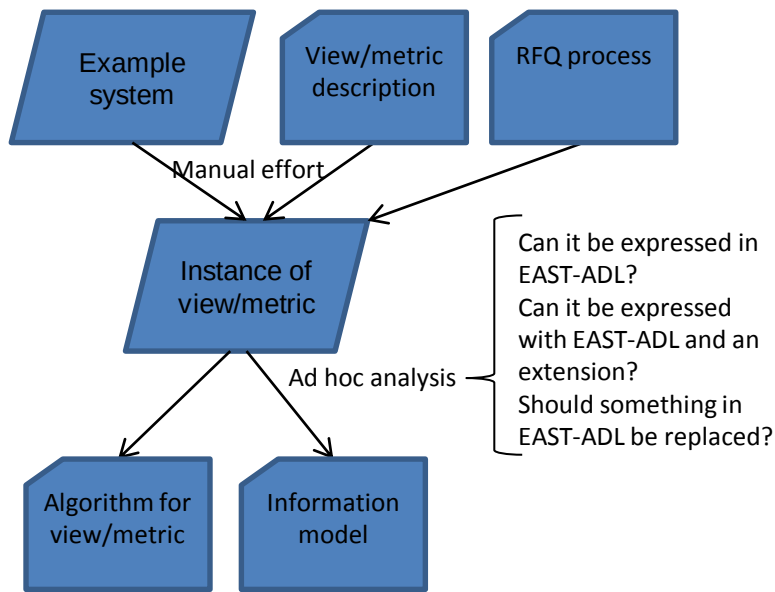
## 2.3 Method

---

Given are views, metrics and scenarios identified in D1.1 [9], AUTOSAR [1], EAST-ADL [3], the information modelling concepts described in Chapter 3 and an example system described in Synligare Deliverable D4.1 [10]. The views and metrics are manually processed one by one, creating a view instance or a metric instance from a given example system. These manually created instances are then analyzed to identify how the specification information should be modelled to support those views and metrics. The modelling of the specification information is compared with the existing information modelling in EAST-ADL leading to at least three cases, namely

- EAST-ADL is adequate
- EAST-ADL should be complemented with an extension
- Some EAST-ADL elements should be replaced

Further, conceptual descriptions of how to automate the generation of the views and metrics are produced by further analysis. This method is illustrated in Figure 2.



**Figure 2. Work flow for contributing in WP2 by defining a view or a metric**

The work progresses iteratively with selection of a view or metric, identification of corresponding specification information modelling and conceptual description of the selected view or metric. As the conceptualization continues, it feeds the work in Work Package 3 (WP3), in which the view or metric is implemented in a tool prototype.

## 2.4 Document Outline

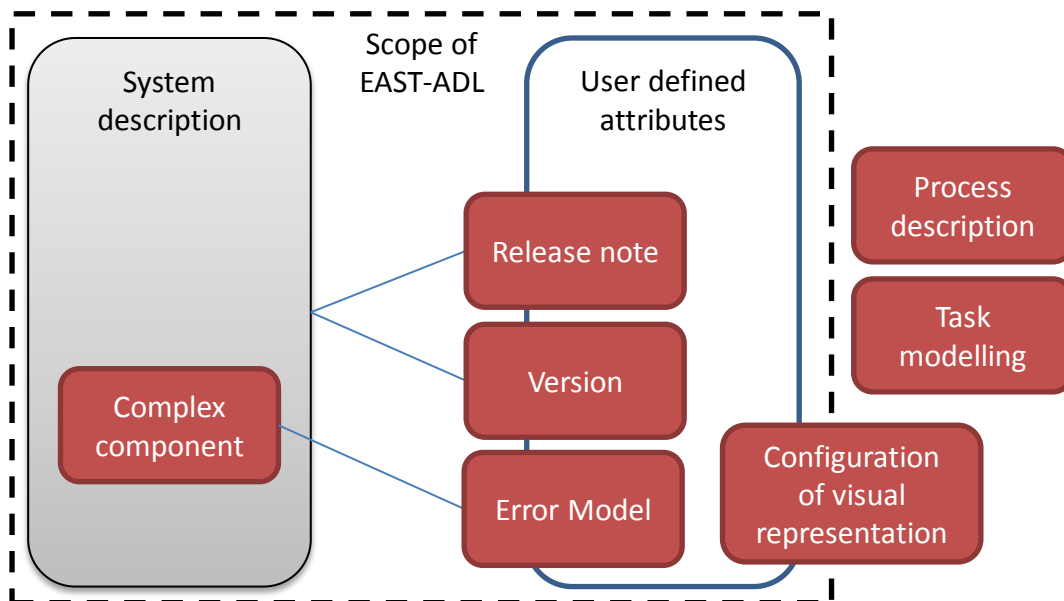
The remainder of this report is structured as follows. Chapter 3 presents information modelling aspects. Chapter 4 presents metrics as conceptual solutions. Chapter 5 presents views as conceptual solutions. Finally, Chapter 6 summarizes the report.

### 3 Information Modelling

This chapter reports on information models identified.

#### 3.1 Overview

Figure 3 gives an overview of the information models reported in this section, with regard to how they relate to the system specification and EAST-ADL.



**Figure 3. System specification vs. work products**

The system description holds information about the system on each level of abstraction in a form that does not make assumption of design or implementation on lower level of abstraction. Consequently, at the time of writing this report, EAST-ADL does not provide dedicated attributes to hold versioning information. Instead, the versioning information is kept as User Defined Attributes, i.e. not required by EAST-ADL. By employing User Defined Attributes, the information models considered in this report stay within the scope of EAST-ADL. Conceptually, the use of User Defined Attributes could later be further studied to identify attributes that should be added to EAST-ADL.

Process descriptions such as the RFQ process are outside the scope of EAST-ADL. Still, these are vital for understanding the activities that are to be supported by the views and metrics that are presented in this report.

The lifecycle of tasks is also vital for understanding activities supported by views and metrics. In particular, there must be some way to represent tasks in various lifecycle phases to describe planning.

When considering concrete views, there is also information to configure the visual appearance of a concrete view. This concerns colour, coordinates, etc. While the information could be held in User Defined Attributes in EAST-ADL, this information has to be in accordance with information modelling assumed by whatever software tool is used to draw the concrete view. It should be noted that software tools that comply with EAST-ADL may use different ways of drawing views.

---

## **3.2 Manufacturer-Supplier Interaction**

---

This section describes processes and process steps appropriate for the interaction between an OEM and a Tier-1. The scenario is that an OEM is developing a system and buys a key subsystem from a Tier-1. The OEM has a system description with an interface to the subsystem and a requirement specification for the subsystem. The Tier-1 has a specification of their subsystem and a model of it. The RFQ-process is conducted to agree on a collaborative development, with its associated cost and time-line. For the Tier-1, this involves identifying the changes that are required on the subsystem to make it fulfill the specifications from the OEM.

---

### **3.2.1 Traditional RFQ-Process**

---

The tradition RFQ-process is shown in Figure 4. The OEM (marked as customer in the figure) and the Tier-1 (marked as supplier in the figure) plays different roles. The OEM owns the system requirements and the system integration. The Tier-1 delivers SW, HW or both with different maturity. During the requirement definition process the OEM and the Tier-1 can:

- Accept requirement
- Reject requirement
- Change requirement

After the contract agreement it is more complex to make changes in the agreed requirements. But almost in all projects this is necessary in some cases.

The introduction of a model based approach will impact this process in a number of ways, which both have pros and cons.

When the models are shared earlier both partners let the other in more closely than otherwise. This is very good from a technical point of view, but it may have commercial implications.

When modelling is made on a deeper level than traditional it is important to choose the right level of details. A lot of requirements could be moved to design, but it must still be implemented and verified.

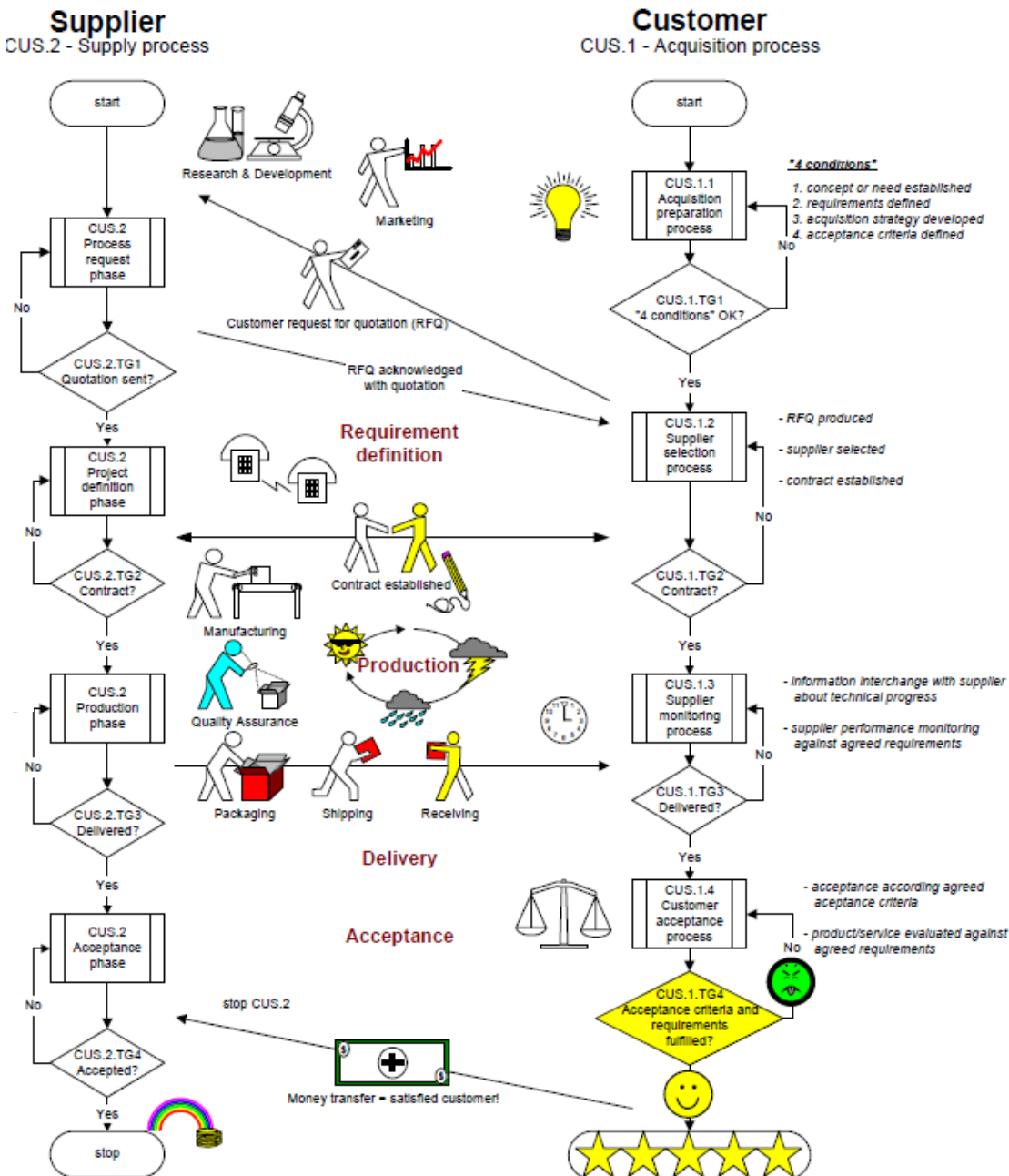


Figure 4. Customer / Supplier process

### 3.2.2 Suggested RFQ-Process

Here, a suggested RFQ-Process that benefits from common information modelling is presented.

Agreeing between an OEM and a Tier-1 can be seen as performed in 3 steps:

1. The OEM defines requested functionality in terms of a specification model which they share with the Tier-1. The content and level of detail corresponds to the OEM requirements.
2. The Tier-1 responds with an updated specification corresponding to the offered functionality. The content and level of detail corresponds to what is available up front and what can be shared.

3. The OEM agrees on the proposed specification and price.

Each step can be iterated until there is a consensus. The minimal content is a Feature Model defining the system content, but typically there will be interface specifications to secure that the negotiated system can be integrated.

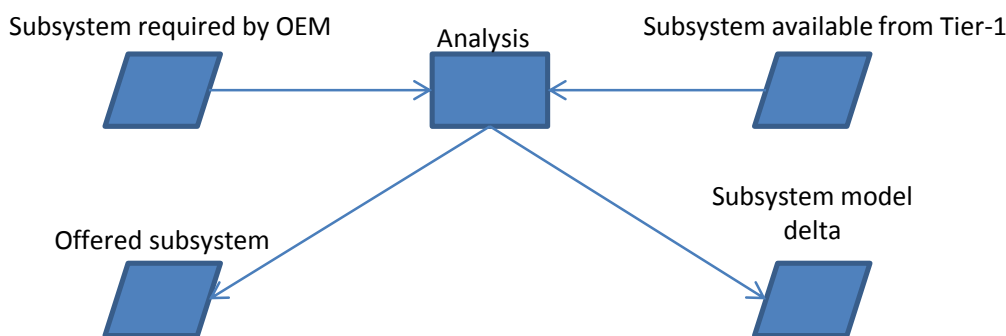
---

### 3.2.3 Analysis of Requirements

---

It should be noted that in step 2 above, the supplier has an analysis activity to perform, to see if and how their existing systems can be adapted to suit the needs of the OEM. All relevant aspects need to be covered, including new or changed functionality, diagnostic functions, safety requirements, etc. A detailed analysis of the requirements of the OEM is typically performed when the agreement has been made that the supplier is commissioned to develop the system.

The analysis involves for the Tier-1 to define a system model delta, corresponding to the additions and changes that need to be applied to the Tier-1's existing solution, as shown in Figure 5.



**Figure 5. Analysis resulting in a subsystem model delta**

At the start of a project to adapt an existing system to a new set of requirements for the system to be employed in a new context, an extensive analysis is to be done, involving collaboration between the OEM and the Tier-1. For this analysis, the necessary information includes the OEM's requirements and the Tier-1's system model. There should be support for analysis steps for functional safety and other important aspects. Reference material, such as templates and checklists should be available.

---

### 3.2.4 Versioning in EAST-ADL

---

In EAST-ADL, it is specified that there should be User Defined Attributes for version, time stamp and status. There is a unique id (UUID) which is tool specific and should not be changed during import of another version of the model, if the element has the same versioning information in both models. Each element is uniquely identified by its path in a hierarchy of elements and its name in its parent element.

---

### 3.2.5 Exchanging Models

---

A typical scenario for exchanging models is

1. The OEM exports an OEM-specific model with versioning-denotation (Vx.y) and current status denotation (explicit status such as accepted/rejected, specified, designed, implemented, verified, etc. or with implicit status such as Satisfy, Realize, VVActualOutcome, etc.)
2. The Tier-1 imports the same model and ignores all elements that are already included based on the versioning information. This includes consciously accepting all elements that have newer version than previously imported.

3. The Tier-1 updates their model with new and changed content and defines the appropriate version and status.

4. The Tier-1 exports their model with regard to the versioning denotation (Vx.y) and the current status denotation.

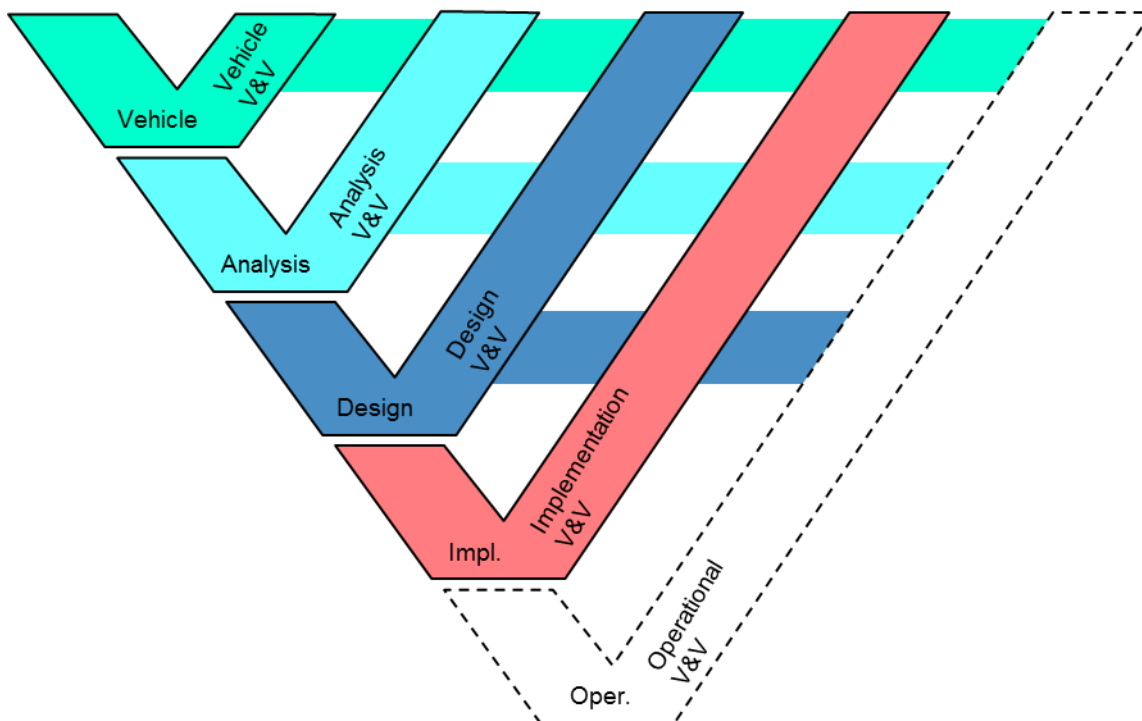
5. The OEM imports the Tier1-specific model and ignores all known elements based on the versioning information. This includes consciously accepting all elements with newer version that previously seen. Similarly, all status that has not been changed is ignored an updated status is automatically imported.

To be able to exchange models, the tools that are involved need to have the following capability:

- Ability to merge models when a new version of (a subset of) the system model is imported with modelling elements that include status elements.
- Ability to show elements that divert in terms of versioning information and allow for manual merging of those elements.
- Ability to differentiate modelling elements and discover if they are different even though the versioning information is the same.

### 3.2.6 Integration Documentation

The OEM and the Tier-1 communicate during the development of the automotive embedded system in several interactions; not only to exchange and review specifications in the RFQ process, but also to synchronize integration, verification and validation activities. In several steps, the embedded system is to be integrated into the larger system (the vehicle). In model-based development, integration can occur early by integrating models before the production software and the production hardware are available.



**Figure 6. Opportunities to integrate and conduct verification and validation**

Typically, such communication of model, software or hardware for an embedded system should be accompanied with release notes, describing the integration configuration in terms of implemented functionality, fault handling, known issues, etc.



A release note typically describes:

- Release date
- Check sums, repository version and repository path
- Implementation status for each function
- Change record since last release
- Verification status for each function
- Interfaces, timing, operational modes, error codes, etc.

An information model regarding integration information and release notes in particular is given in the following. Integration information models have the purpose to define the information necessary to enable exchange of release notes and integration activities.

---

#### 3.2.6.1 Work-Phases when Eliciting Integration Information

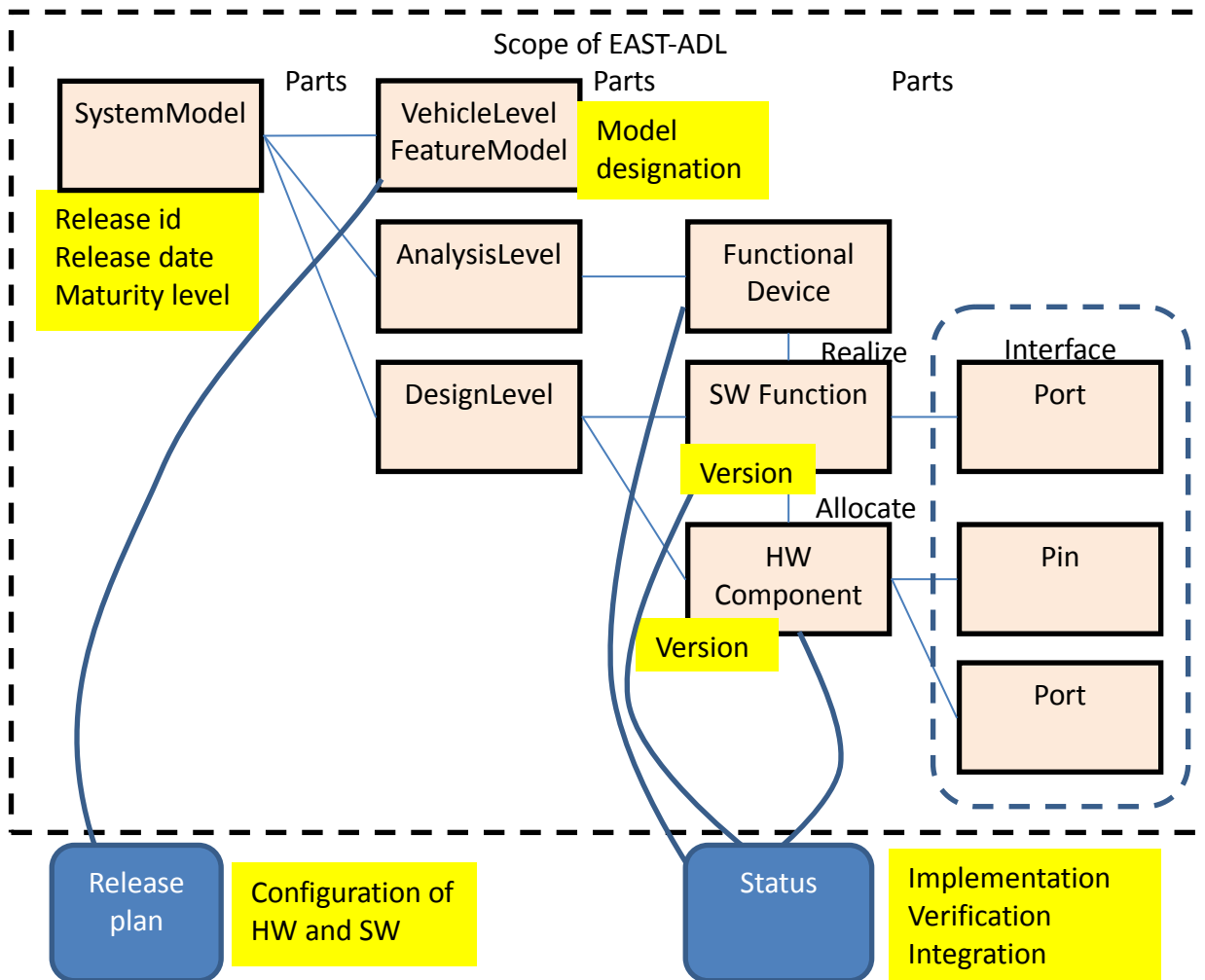
---

To have integration information available the following processes must have reached sufficient status:

- Release planning
  - Configuration of HW and SW for the release must be known
  - Deadline for the release must be known
- Versioning
  - System, HW components and SW components must have versions
- Status reporting
  - Implementation, verification and integration status must be known for functions, HW components and SW components

Integration information is elicited by gathering information from stakeholders, such as customer, system team, integration team, software team and hardware team. If any of the needed information for the software and hardware components is not in the intended container, it should be synchronized with information in the system model.

Figure 7 shows a sketch of nodes, relationships and attributes that are needed in the information model such that the information is sufficient to fill in a release note.



**Figure 7. Information model sketch for containing information relevant to a release note**

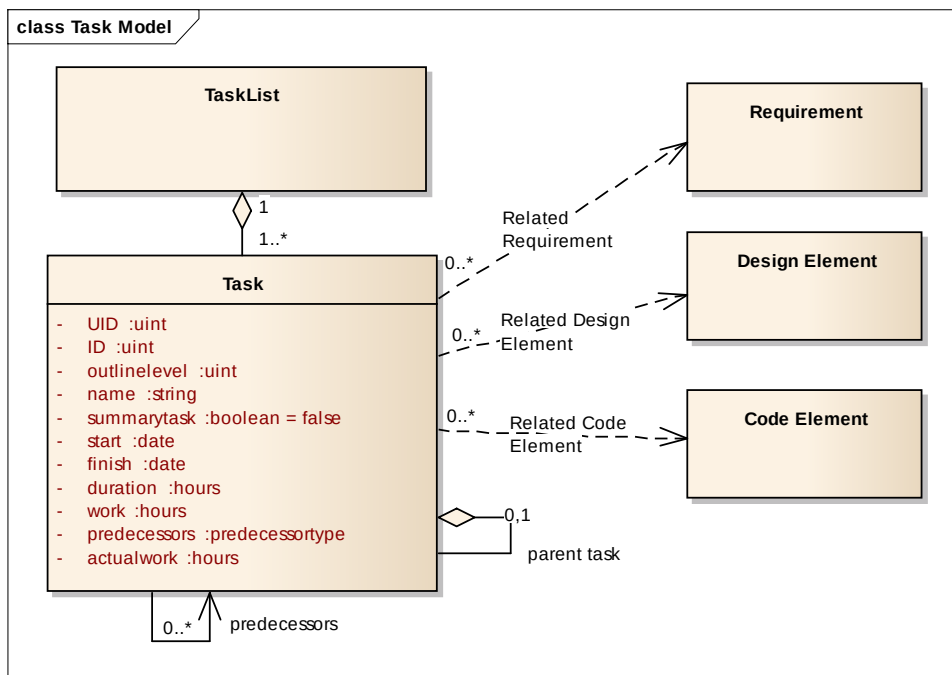
In Figure 7, filled rectangles with a border are modelling elements defined in EAST-ADL and as a modelling element can have parts, that is represented by a line to the right. The relationships Realize and Allocate from EAST-ADL are shown by vertical line. The yellow rectangles without border show information that can be represented by using User Defined Attributes. The rectangles with rounded corners cannot directly be represented in EAST-ADL. In this case, the release plan is not a part of the specification or the system model. A modelling element for representing status information might not be necessary if it can be generated as a view that aggregates metrics from status information kept as User Defined Attributes in the individual SW Components, HW Components and Functional Devices.

### 3.2.6.2 Comparison with EAST-ADL

As can be seen in Figure 7, EAST-ADL contains most of the necessary modelling elements, but it does not specify all the necessary attributes, but rather supplies the User Defined Attribute option for representing attributes that are not explicitly specified in EAST-ADL. Further, EAST-ADL does not specify modelling elements for plans. Considering the concept of a status summary, aggregated from the attributes of a set of modelling elements, this should not be a modelling element in itself but rather be a function supplied by a software tool operation on information represented in EAST-ADL.

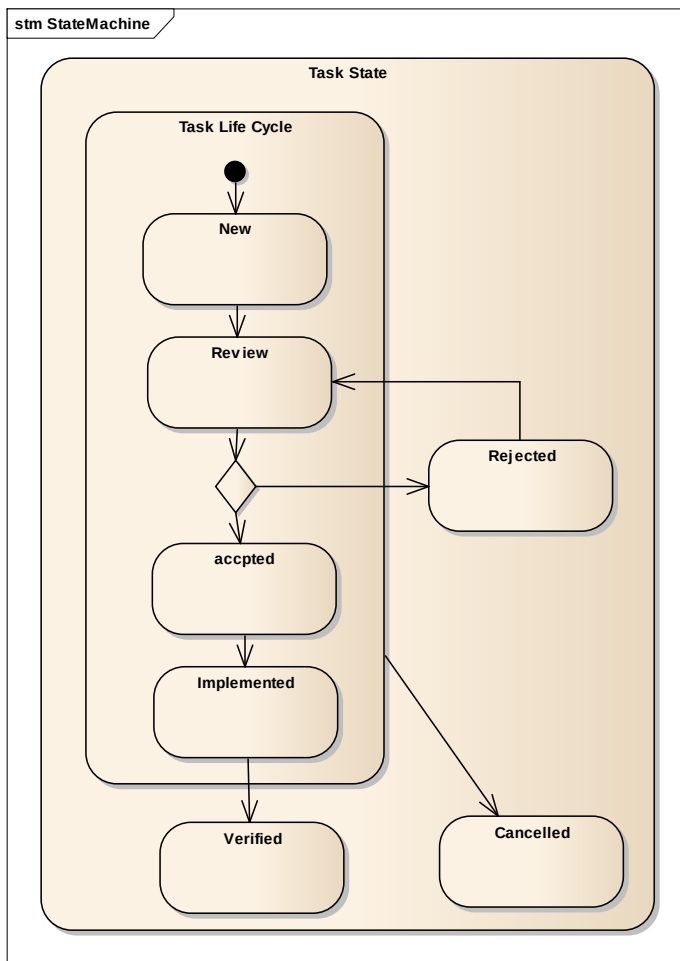
## 3.3 Task Model

For the sake of understanding the activities that views and metrics are to support, this section considers a model for tasks and the lifecycle of tasks. The information model for tasks is useful for planning and scheduling, as well as for tracing progress. Tasks are not part of the system description, and are not included in the scope of EAST-ADL. The task model includes attributes, dependencies and other definitions see Figure 8.



**Figure 8. Relevant information for keeping track of a project task**

By defining an XML schema for the task model, project status information can be exchanged between tools. Each task has a life cycle according to Figure 9.



**Figure 9. States useful to track the progression of a task**

### 3.4 Tooling Support EAST-ADL models

Figure 10 shows the principles for data exchange between the tools: SystemWeaver (SW), EATop (ET) and Enterprise Architect (EA).

The figure contains an example. Consider that an OEM, in this case Volvo, would develop an adaptive speed limiter system (ASL) and collaborate with a Tier-1, in this case Autoliv, on developing a traffic sign recognition subsystem (TSR).

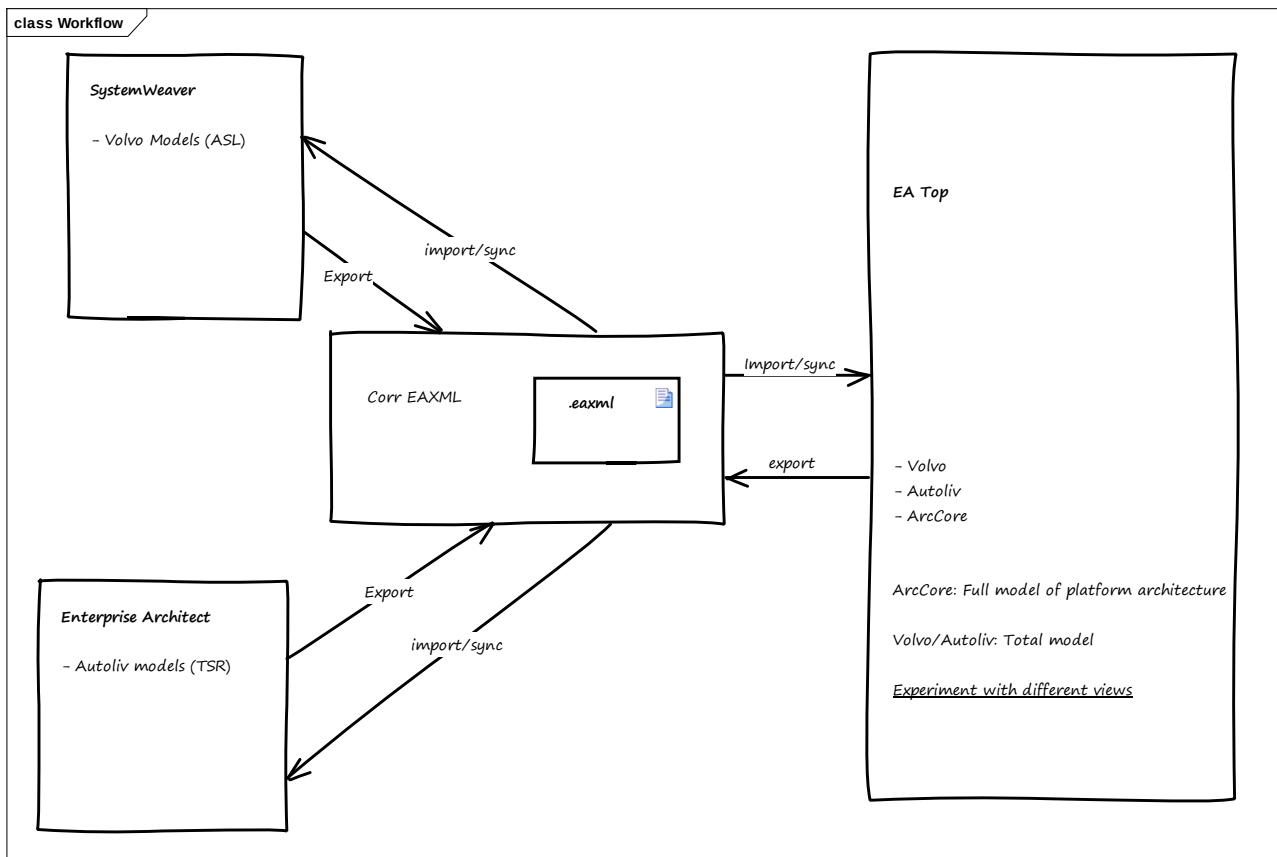
Their respective tools contain their own models, stored separately on each part's configuration management tool. The model exchange functionality is implemented by each tool vendor. There is a core model for the system and there is a tool to access it.

This means that the OEM (e.g. Volvo) has the core system model for ASL. It is exported and can be used by Tier-1 (e.g. Autoliv), but it is normally not changed. The Tier-1 can make some updates in the interface and export it back to the OEM, and this can be compared to the core model by the OEM.

The Tier-1 works with the model for the TSR subsystem.

The Tier-1 and the OEM may choose to export only parts of the model. This is fine as long as the interfaces and the common function are synchronized.

All partners: OEM, Tier-1 and Tier-2 (ArcCore, Systemite) can work in EATop with the full model or parts of it.



**Figure 10. Data exchange model**

Each exchange tool must be able to handle the following:

- Import of a new model
  - o created in any EATool that can produce EAST-ADL models
  - o saved in .eaxml-format.
- Previously imported models could only be synchronized during import. It shall be possible to do this in one of these methods:
  - o Either each change can be monitored by the user and be individually accepted/rejected
  - o Alternatively all updates can be shown and the user can do the total import or stop it.

Other demands:

- The unique UUID which uniquely identify objects in the core tool shall be stored together with the other data
- An export from an imported .eaxml-file shall produce an identical .eaxml-file
- It shall be possible to diff between the old model and the updated
- An object that no longer is a part of the new (imported) model is not removed during synchronization. A warning is printed and it is up to the user to remove manually.

Limitations:

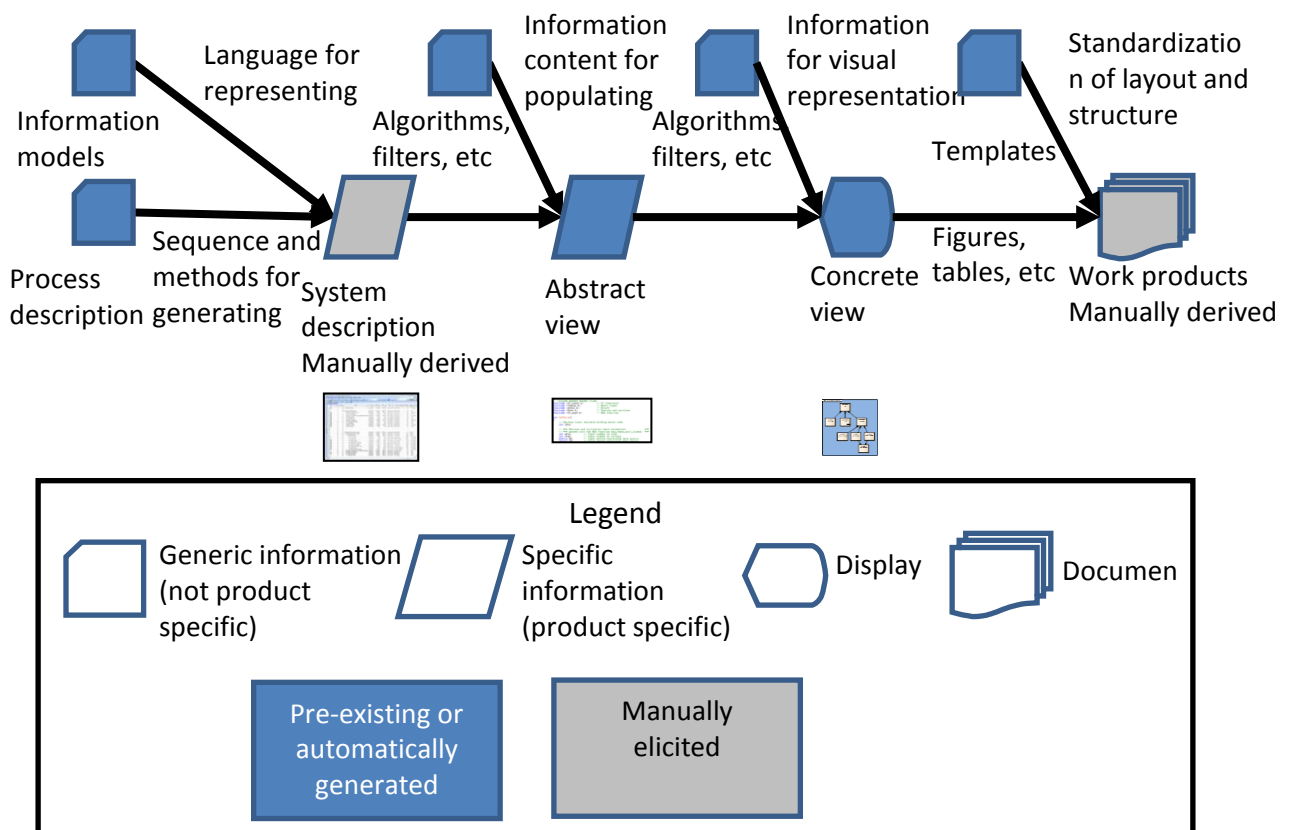
- No support for variability
- No support for versions

- Import/Synchronization is done manually. This means that the imported model may not match if synchronization is done seldom.
- No support for error model in EA
- EAST-ADL uses the object name and not the UUID as identifier. This means that the UUID is not used. In EA import function two functions are used handle this
  1. A warning is printed out if two object that have the same name
  2. A warning is printed out if two objects have the same UUID

Regarding parsing of .eaxml-files that do not fully comply with the corresponding XML-schema, the EAST-ADL schema eastadl\_2-1-12.xsd [4] is used for validation of the .eaxml-file. If there is an error a warning is printed out, but the model is still imported (for all items that match).

### 3.5 Tooling Support Graphical Model Exchange

A lot of the system modelling includes Diagrams which shows the most important views of the system. These diagrams are not supported in EAST-ADL. In order to exchange core graphical information a simple Diagram XML format is proposed. The exchange of this information is done in the same time as the EAST-ADL model exchange.



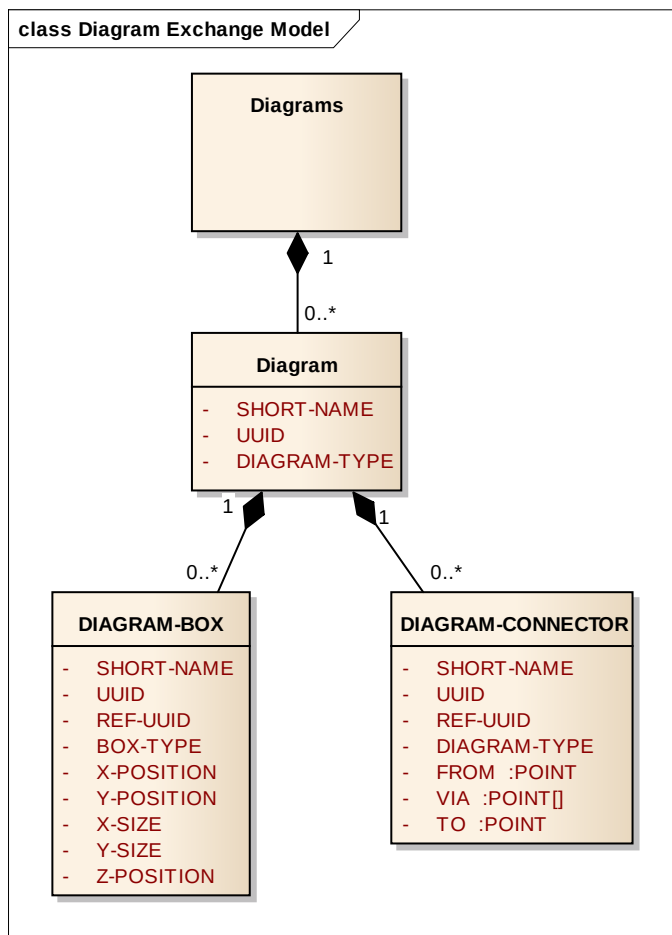
**Figure 11. Context for information models and concrete views**

Figure 11 shows the context for the information model and the concrete view. The Diagram XML format is to be used to record the figures that come from either generating a concrete view by applying algorithms, or from editing a concrete view.

A key aspect to be considered in the context of concrete views is to make views recognizable. The understanding and overview of an engineer with respect to a system is much aided if the engineer can recognize the system's structure in how the concrete view is shown. Therefore, views that contain the same information should ideally look similar. A view should not change from day to

day unless the underlying information has been altered so much that a different structure is modelled.

### 3.5.1 Diagram Exchange Model



**Figure 12. Diagram exchange model**

Each exchange tool must be able to handle the following:

- Import of a new diagram.
- Previously imported diagrams can be synchronized during import. It shall be possible to do this in one of these methods:
  - o Either each change can be monitored by the user and be individually accepted/rejected
  - o Alternatively all updates can be shown and the user can do the total import or stop it.

Other demands:

- The REF-UUID must point to a previously imported (or created) EAST-ADL element
- The diagram elements can be modified in the importing tool. For example more details like colouring etc. can be added. This information is stored within the tool and will not be a part of the exchange. If elements and connectors are moved or resized or this will be exported.
- The unique UUID which uniquely identify objects in the core tool shall be stored together with the other data

- An export from an imported .xml -file shall produce an identical .xml-file
- It shall be possible to diff between the old diagrams and the updated
- A diagram object that no longer is a part of the new (imported) model is not removed during synchronization. A warning is printed and it is up to the user to remove manually.

Limitations:

- No support for variability
- No support for versions
- Import/Synchronization is done manually. This means that the imported model may not match if synchronization is done seldom.
- The UUID as the identifier. This means that the UUID is not used. In EA import function two functions are used handle this
  1. A warning is printed out if two object that have the same name
  2. A warning is printed out if two objects have the same UUID

Regarding parsing of .eaxml-files that not fully comply with XML-schema, the schema eastadl\_2-1-12.xsd [4] is used for validation of the .eaxml-file. If there is an error a warning is printed out, but the model is still imported (for all items that match).



---

**4 Metrics**

---

Various metrics on engineering documentation can be used to assess the product's properties or the development progress. In Synligare, we are investigating metrics that can assist the collaborative aspects of development, with particular focus on safety aspects. In general, all metrics that assist planning and follow-up engineering work are beneficial from a safety perspective, since faults caused by lack of resources and lack of time are impeded.

A metric is calculated on the basis of an entire system specification or parts thereof. Examples of delimitations include

- Architecture part - The metric concerns a particular architecture or parts thereof
- Requirement set - The metric concerns a particular requirement set
- Project - The metric concerns a particular project

A hierarchy of metrics based on the structural hierarchy of the system (system, subsystem, etc.) is conceivable. Depending on character, metrics can in a tool or in a report be visually represented as lists, flags and colors in addition to plain numbers.

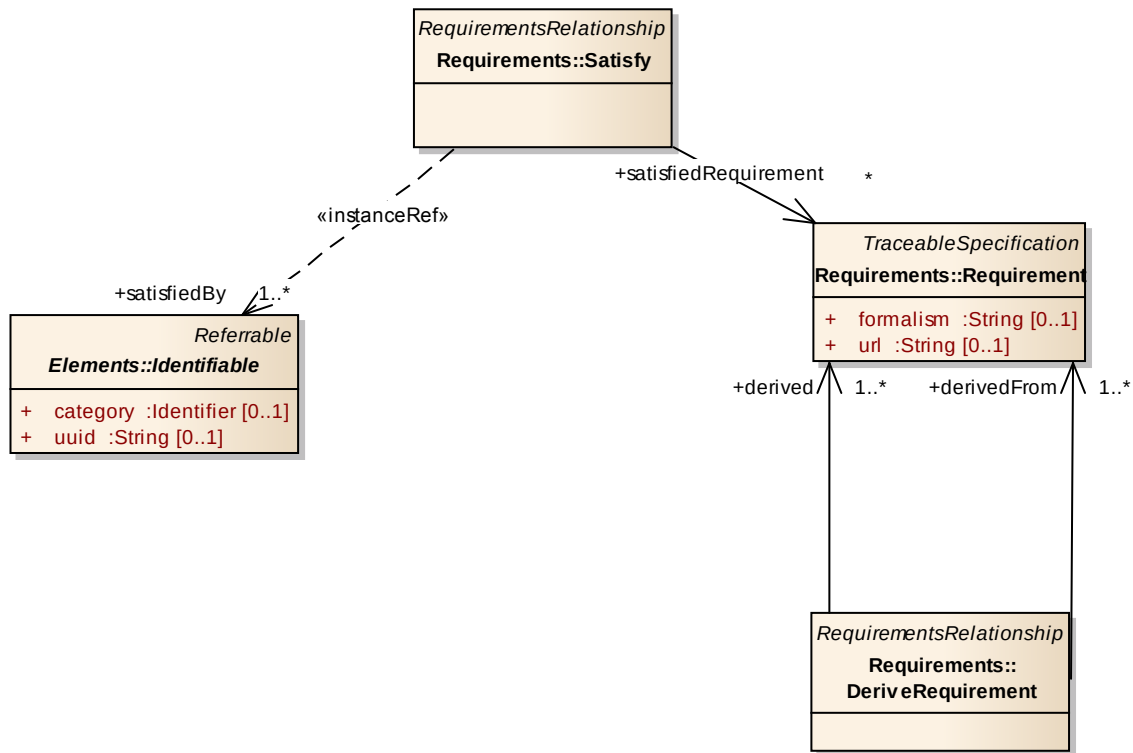
Two kinds of metrics will be discussed below, progress and product metrics. Development progress can be assessed based on the status of the model or based on assessments of actual status that is reflected in the model.

---

**4.1 Requirement Progress**

---

Assuming that requirements are used to specify what the product shall do, they can also be the basis for assessing progress. Progress of requirement allocation is measured as the fraction of requirements allocated to architectural elements.



**Figure 13.** Requirements can be considered allocated when there is a satisfy link, or its derived requirements have a satisfy link.

Because requirements may be derived to more specific requirements, a requirement with one or more derived requirements is also considered as allocated.

Progress of requirement implementation is measured as the fraction of requirements that are implemented in the product. This criterion is not well-defined, as it may concern requirements with a final design, final software specification, final code, compiled code, etc.

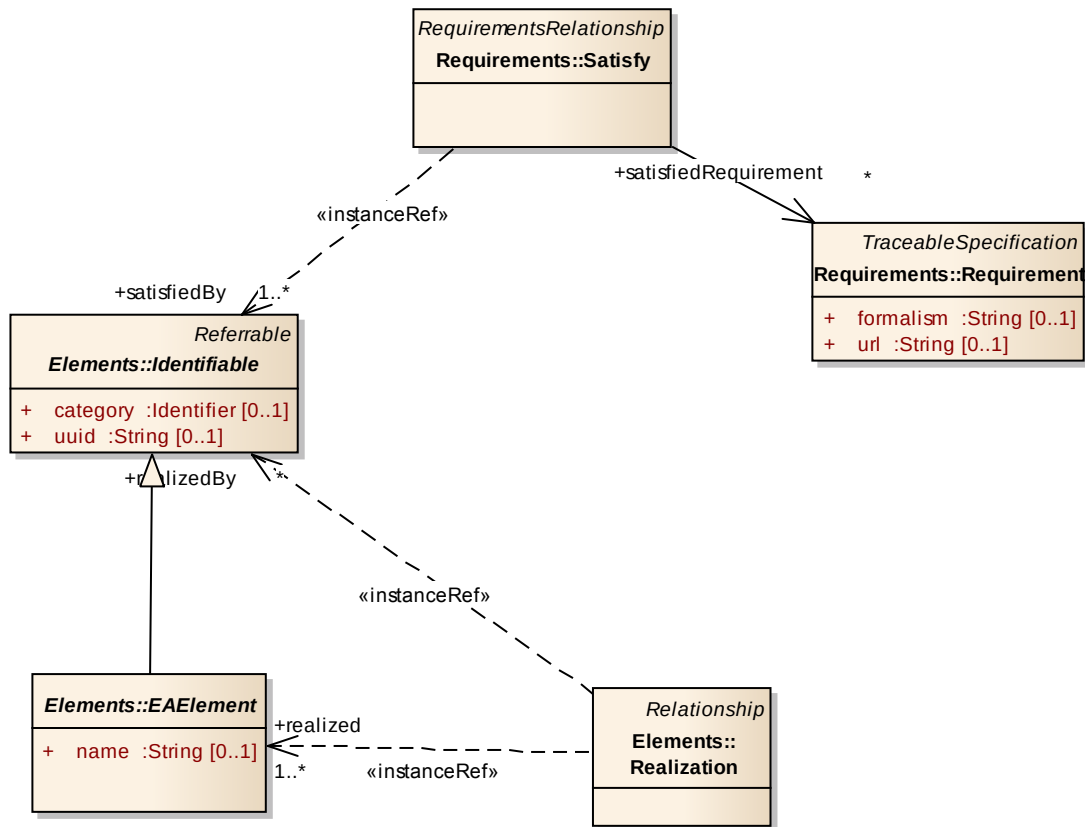
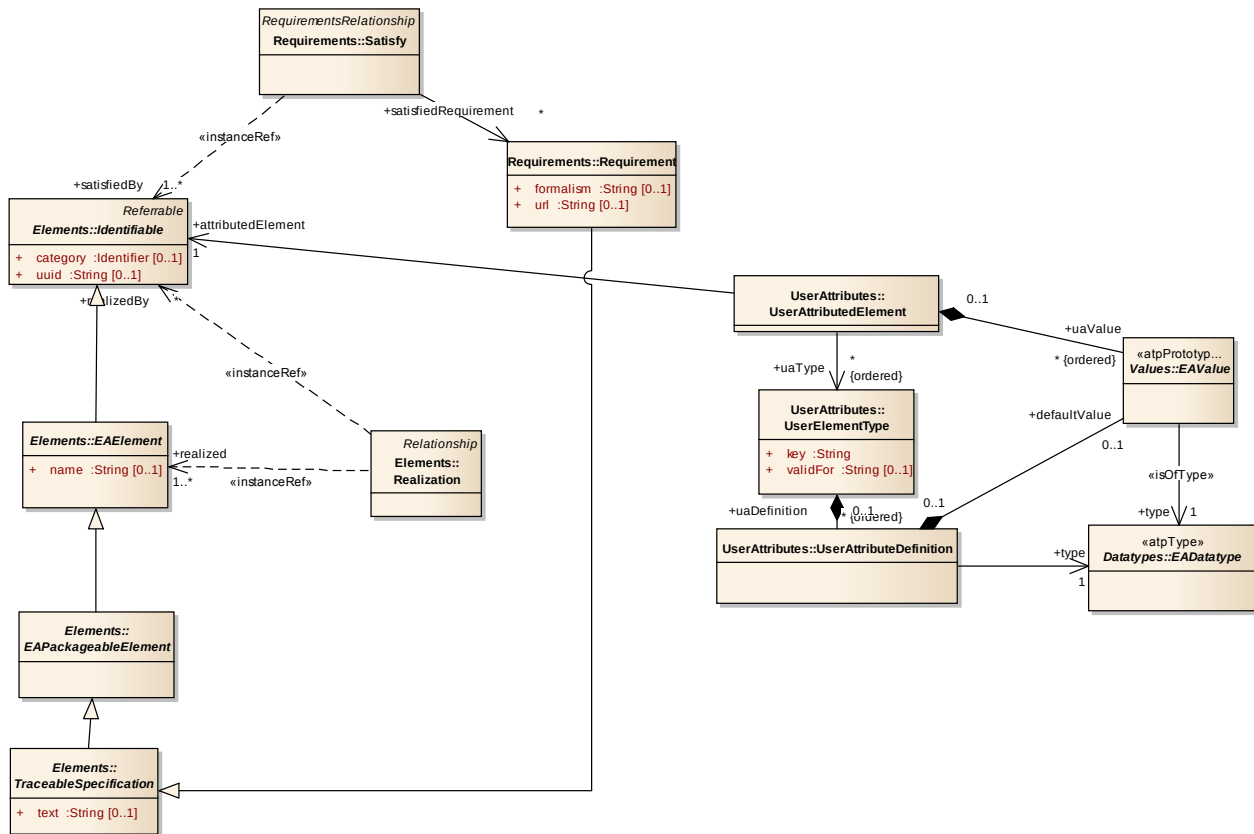


Figure 14. Requirements can be considered done when they are implemented, i. e. their target architectural element is Realized.



**Figure 15. User Attributes can be used to put flags on model elements**

On computing this metric, the model can be inspected for one of the criteria mentioned, or a specific flag that is manually set by the engineer is used as a basis.

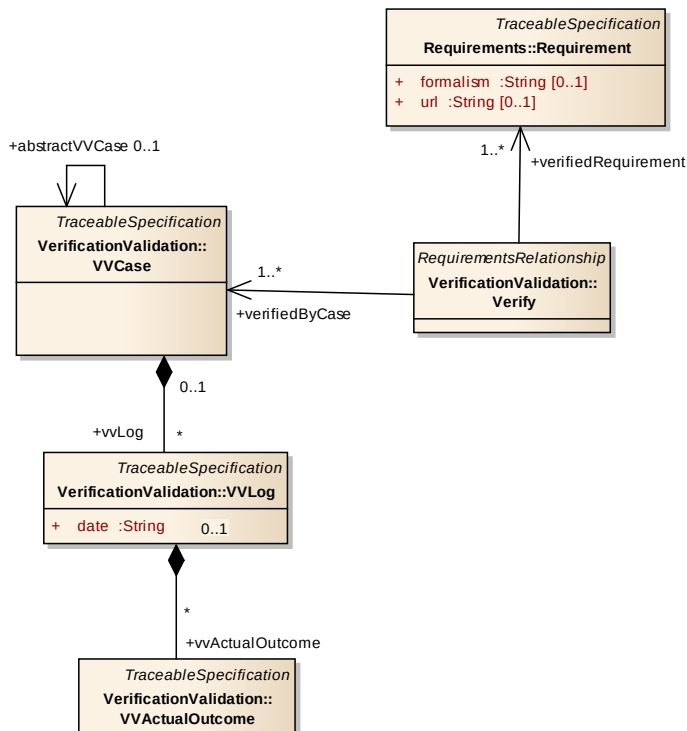
## 4.2 Realization Progress

The EAST-ADL model (1) provides means to relate model elements to its abstract counterpart. For example, a function or ECU is linked to the Vehicle Feature it is realizing. In the end all Vehicle Features should be realized down to the Software Component level. On computing realization progress, one must decide which criterion to use, for example realization in software, function or hardware architecture. One must also be aware that it is not possible to know how many concrete elements are required for realizing an abstract element. For this reason, an alternative approach is to use a specific flag which is manually set when a feature is deemed to be realized.

Figure 14 above shows the Realization relation between artifacts, and Figure 15 shows how to attach flags using the user attribute mechanism.

## 4.3 Verification Progress.

In EAST-ADL, the verification specification VVCase is linked to requirements. For each requirement one or several verification cases can be defined, each of which may pass or fail. Figure 16 shows how requirements and verification are related. Note that there is an abstract VVCase defining the test or analysis steps required, and an abstract VVCase that holds one or several verification results.



**Figure 16. Verification specification for Requirements**

Verification progress may thus concern the fraction of requirements that are verified or where a verification method is defined. If multiple verification cases are defined per requirement, the fraction may be based on verification cases instead. In case verification is done, a useful metric is the fraction of passed vs. failed requirements.

#### 4.4 Safety-related Progress Metrics

The ISO26262 functional safety standard [6] defines a number of engineering artifacts that shall be defined in the context of its Item. One of these artifacts is the safety goal, which is the starting point for organizing safety requirements. Examples of metrics include the fraction of Items with safety goals defined or the fraction of safety goals with functional safety requirements. Because it is system specific, it is not possible to know how many safety goals or safety requirements are needed. It is still a useful metric, since the amount must be non-zero.

Another safety-related metric is the fraction of functional safety concepts that are matched by a technical safety concept. This can be computed based on derivation relations between functional and technical safety requirements combined with realization relations between analysis functions and design functions making up the architectural aspects of the functional and technical safety concepts, respectively.

Figure 17 shows how the FunctionalSafetyConcept element identifies a set of Requirements. These requirements together with the elements to which they are related with a satisfy relation makes up the FunctionalSafetyConcept. The corresponding SafetyGoal is found through the Derive relation to the requirement identified as a safetyGoal.

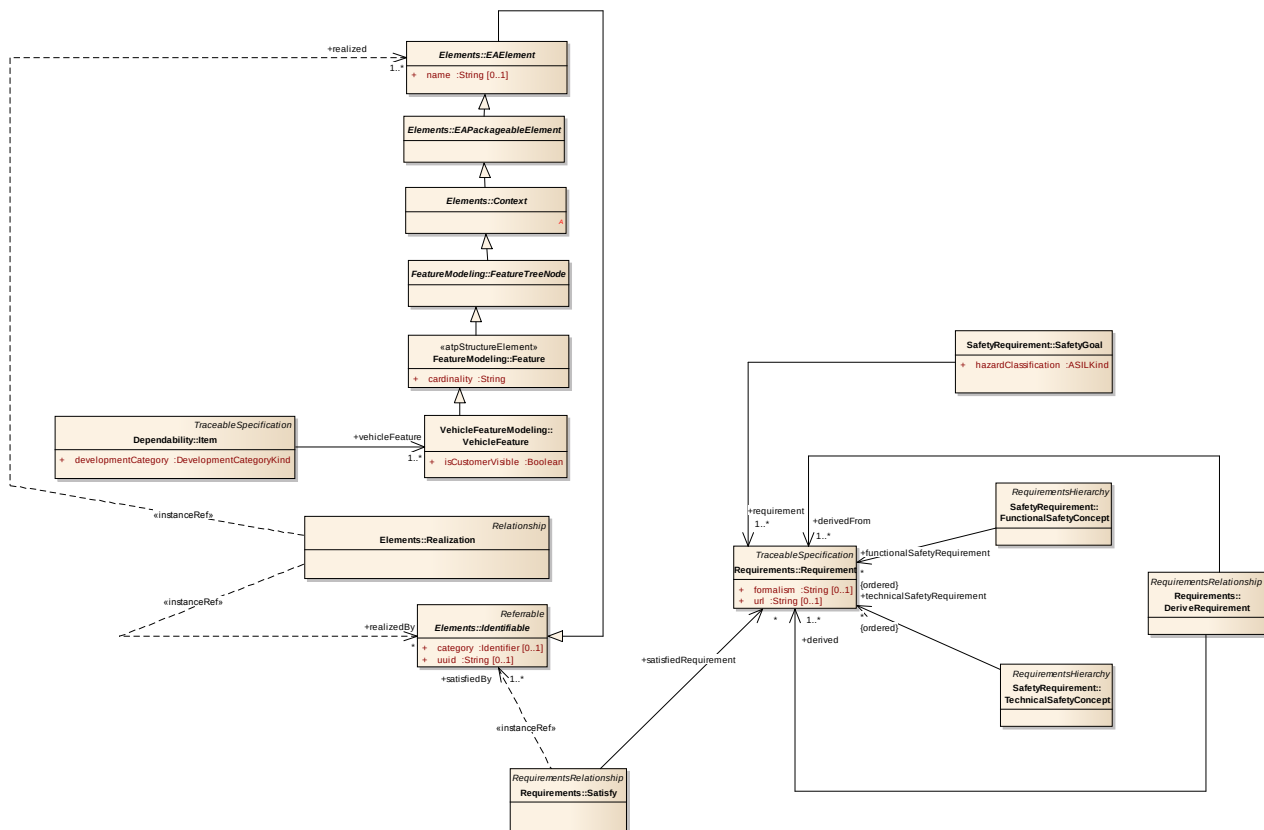
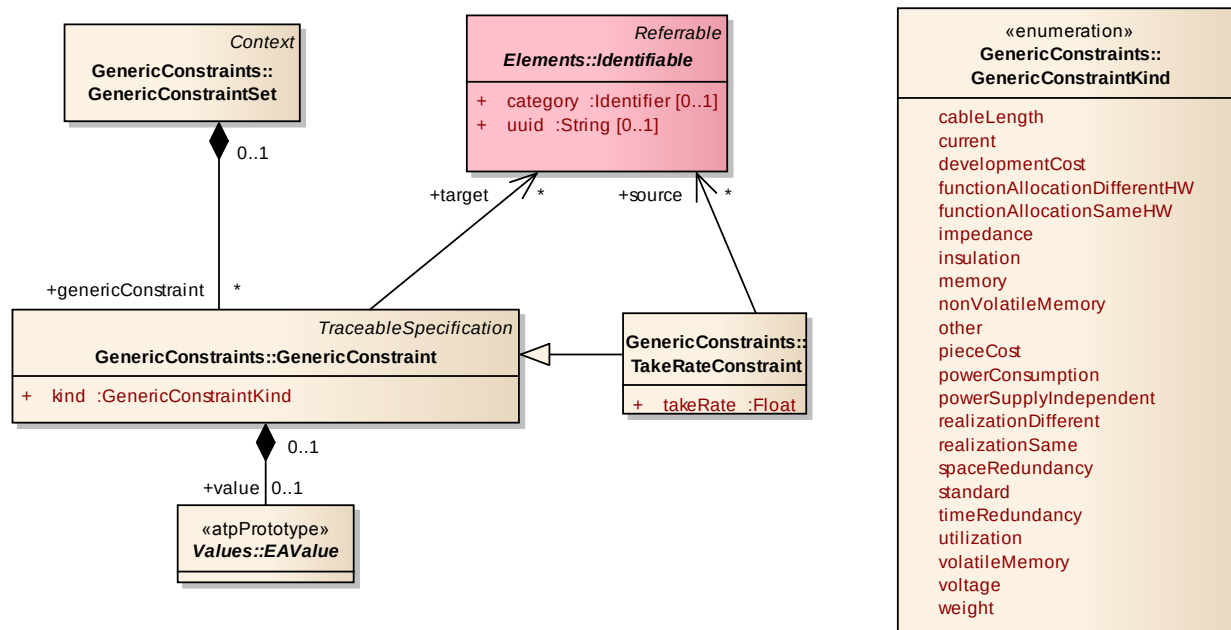


Figure 17. Elements and relations related to Functional and Technical Safety Concept

## 4.5 Product Metrics

As opposed to progress metrics, product related metrics are typically absolute. One metric that is relevant for strategic decisions regarding development rigor or priority, is the number of manufactured vehicles that are impacted by a certain engineering artifact. This can be computed based on the take rate annotations of vehicle features realized by the engineering artifact considered. Similarly, total revenue or cost can be computed by combining take rate information with financial annotations.

Figure 18 shows how GenericConstraints of kind pieceCost can be used to define cost. The TakeRateConstraint defines with an absolute number how many elements are produced of the target, if the source is empty. If a source is also identified, the take rate is a fraction.



**Figure 18.**  **GenericConstraint** can be used for annotating elements with a cost and take rate.

Various dependability metrics are of course safety relevant too. These include failure rate and other hardware architecture metrics.

#### 4.6 Identifying Context.

In general, it is necessary to be aware of context when computing metrics based on fractions. The AUTOSAR/EAST-ADL notion of package is one possible context. Another is to consider a certain feature, function, hardware component and compute a fraction among the related elements.

## 5 Views

In this section, views that have been processed in WP2 are described.

### 5.1 Feature-based Architecture Views

These views show the System and Software Architecture with regard to elements that realize a certain feature.

Architectural elements, such as functions, sensors or ECUs have the responsibility to realize one or several Vehicle Feature. To identify which architectural elements are related to a certain Feature, it is useful to have a view that hides other elements or highlights the relevant elements.

#### 5.1.1 Example View

Figure 19 shows how elements that realize a selected feature are highlighted.

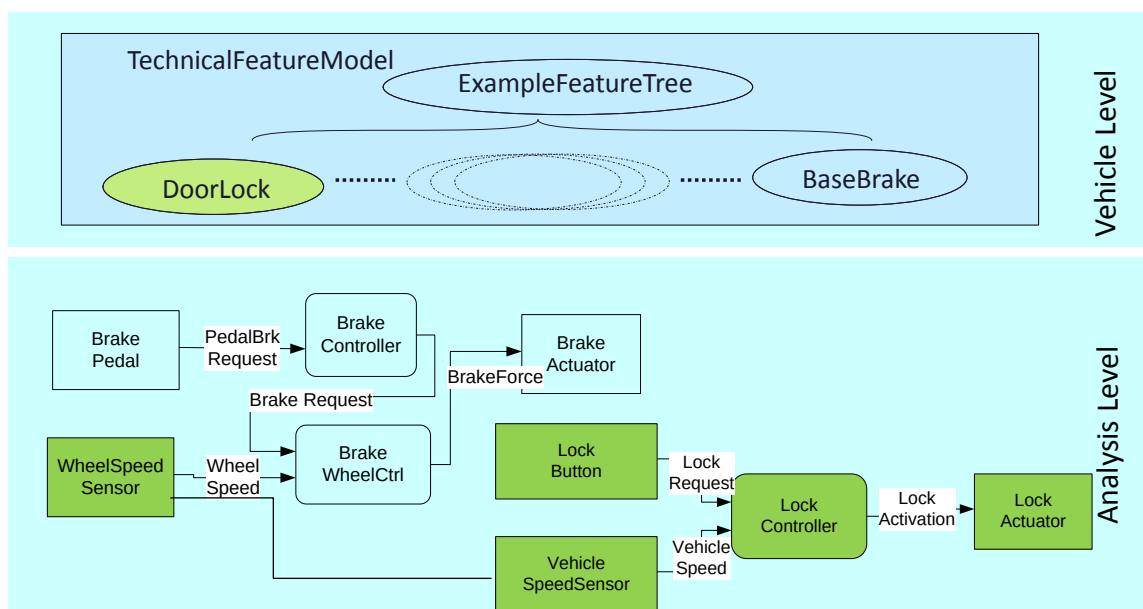


Figure 19. Highlighting elements that realize a specific feature.

#### 5.1.2 Related Modeling Concepts

The elements that realize a specific feature are found by following the Realize relations from the feature to elements on the analysis, design or implementation level. Figure 20 shows the relevant metamodel excerpt.



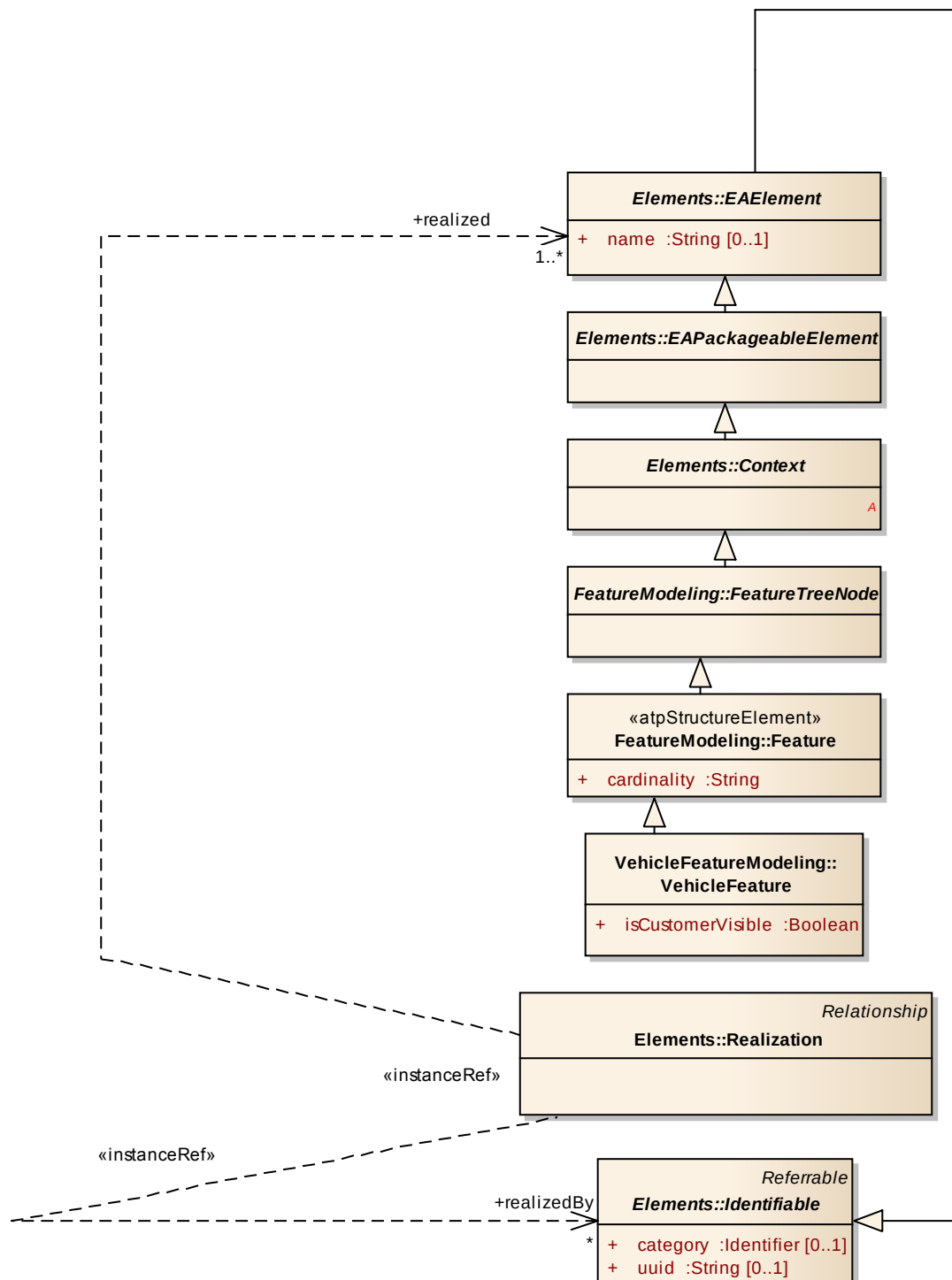


Figure 20.  Realization relation between Identifiable and EAElement.

## 5.2 Requirements vs. Architecture

Requirements are normally related to specific parts of the functional, hardware and software architecture. It is expected that the allocation of requirements can make the requirement set easier

to understand and manage. Views that present the requirements according to the system hierarchy are thus beneficial.

### 5.2.1 Example View

This view shows requirements as they are allocated to architectural components.

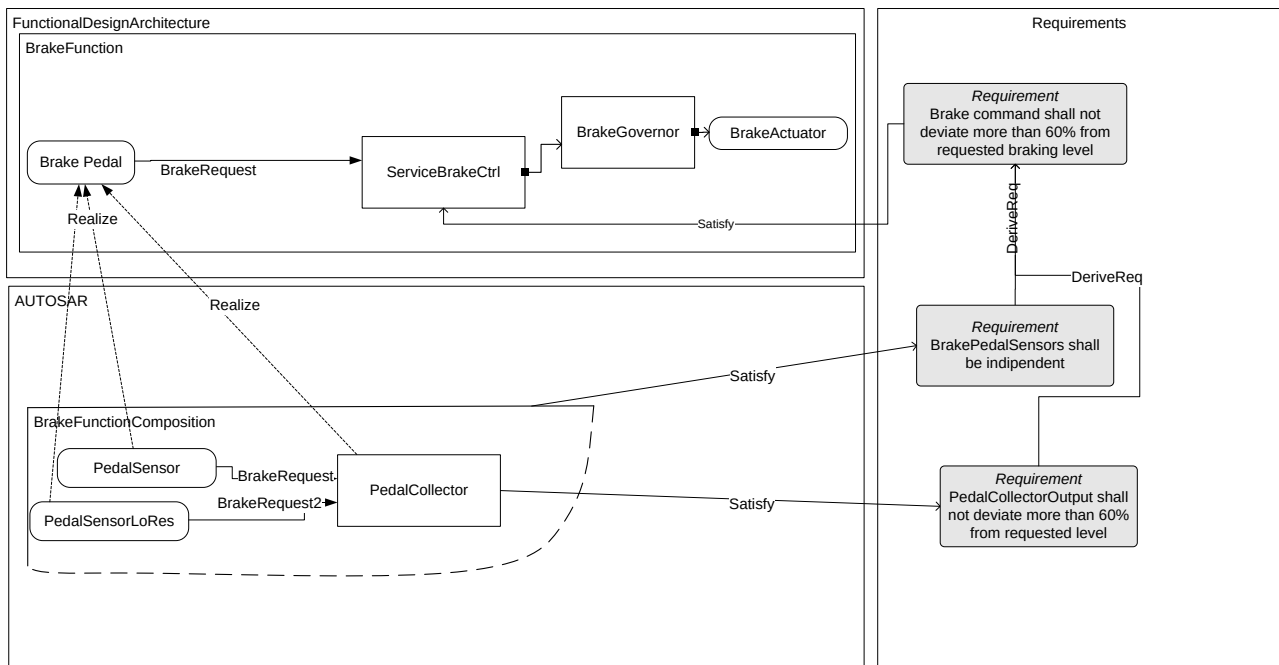


Figure 21. Example of requirements and architectural elements shown together.

### 5.2.2 Modelling Concepts

Figure 13 shows the Satisfy and requirement element together with identifiable, which are the relevant elements for the discussed view.

## 5.3 Software Components, Ports and Signals

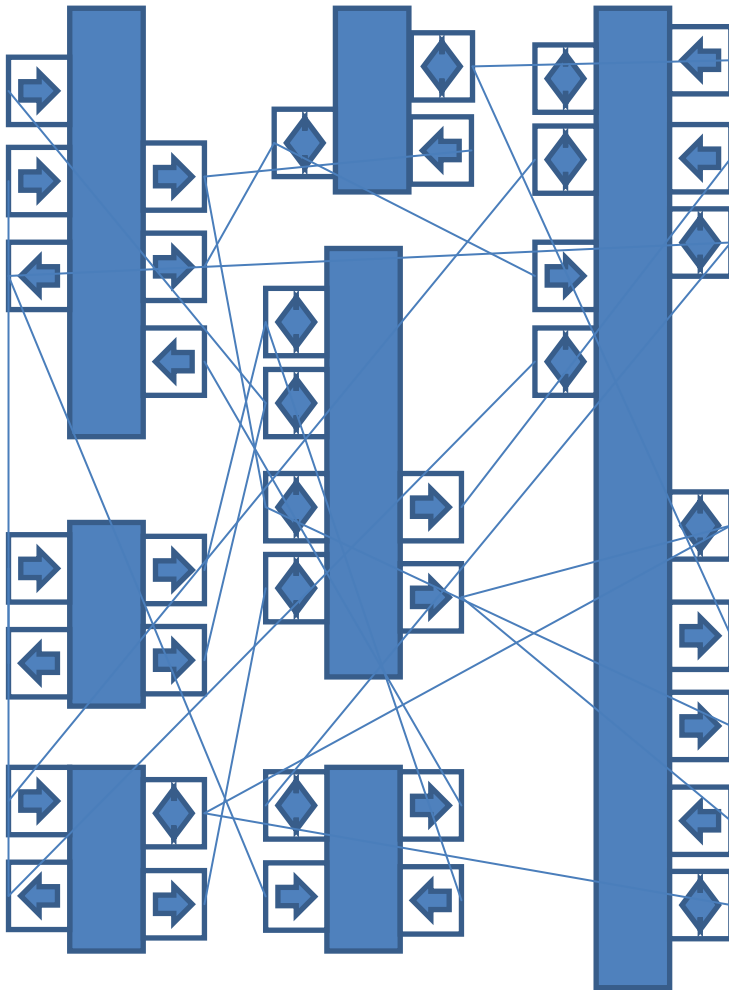
This view is there to show the System and Software Architecture, in particular how elements realize a feature.

On the analysis, design and implementation level, functional and software components (for example a set of communicating applications on an AUTOSAR platform) are in some activities modelled as components with ports and connectors to represent how data may flow. With many components, many signals and various types of ports (in, out, bidirectional) the complexity easily increases, leading to a system that is difficult to make sense of, unless something is done to provide overview.

### 5.3.1 Example View

Consider the concrete view in Figure 22, consisting of seven components represented as vertically aligned rectangles, decorated with ports represented by squares. Each port has a symbol to show which type of port it is. The symbol of an arrow towards the component is an input port. The symbol of an arrow away from the component is an output port. The symbol of a diamond is a

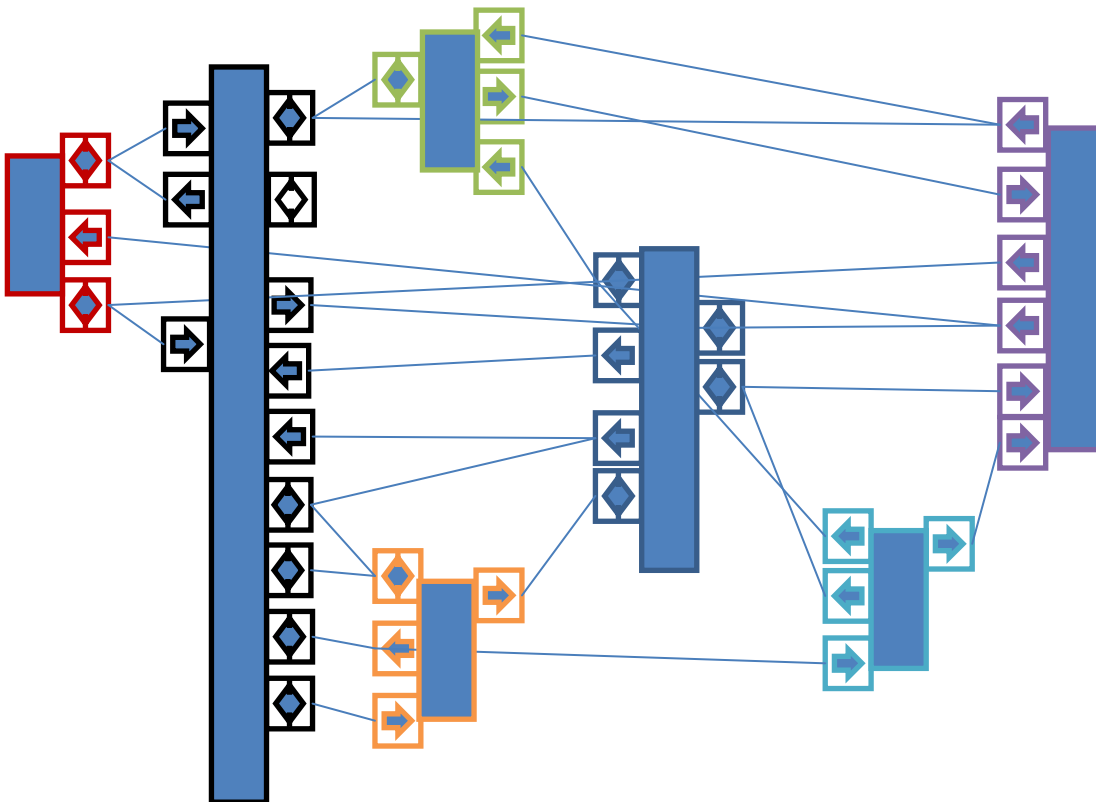
bidirectional port. Between the ports are lines to show connectors, i.e. predesigned possibilities to pass data. The line always starts and ends at the end of the port symbol that is away from the component.



**Figure 22.** Components, ports and connectors

Consider the various dimensions of freedom to exploit to lend more overview to the software component view.

- Color – for example color the components and associated ports with one color for each component.
- Placement of components – for example put components that are connected by signals close to each other so that signal lines become short in the graph.
- Order and placement of ports – for example let ports come in such order that signal lines do not cross each other and place the ports on the side of the component for the direction that the signal will go



**Figure 23.** Illustration of ports and connectors with improved layout.

Figure 23 shows the same view as Figure 22, but here the dimensions of freedom have been exploited to bring some order. There are improvements and these have made it possible to follow most of the signals and to identify ports that are not connected to any signal. While the example shows that much can be done to lend overview to a complex view, it also shows that it is very difficult to make it completely clear. Further, if one has learned about the software by looking at Figure 22 might not recognize it in Figure 23. The value of recognition in getting and overview is not to be under-estimated.

Conceptually, an algorithm could calculate the necessary information based on a set of rules every time the concrete view is to be shown. The problem with relying on an algorithm is that a small change in the information of the view, the algorithm could end up drawing something that is not directly recognized as the same system. A strategy is needed for when to completely redraw the view and move components around, and when to leave it as it is and just mark the changes.

### 5.3.2 Relation to EAST-ADL

EAST-ADL contains the modelling elements for components, ports and signals. However, EAST-ADL does not specify the information elements to hold the necessary information to show the view. For example, EAST-ADL has no attributes for the position and size of a rectangle for the component. To show a concrete (decorated) view, information that dictates the visual appearance of the view has to be represented using some other information than EAST-ADL.

## 5.4

## 5.5 Item Definition

ISO26262 use the concept of Item to define the “system or array of systems to implement a function at the vehicle level, to which ISO 26262 is applied”. Elements of an Item are hardware (ECUs, sensors, cables, ..) and software (functions, connectors, software components, ...) of various kinds and represented on various abstraction levels. During Item definition it is therefore useful to show the elements concerned to secure the right boundary. In an FunctionalDesignArchitecture, HardwareDesignArchitecture or SoftwareComponentTemplate, it is thus useful to hide all elements that are not included in the Item that is inspected.

### 5.5.1 Example View

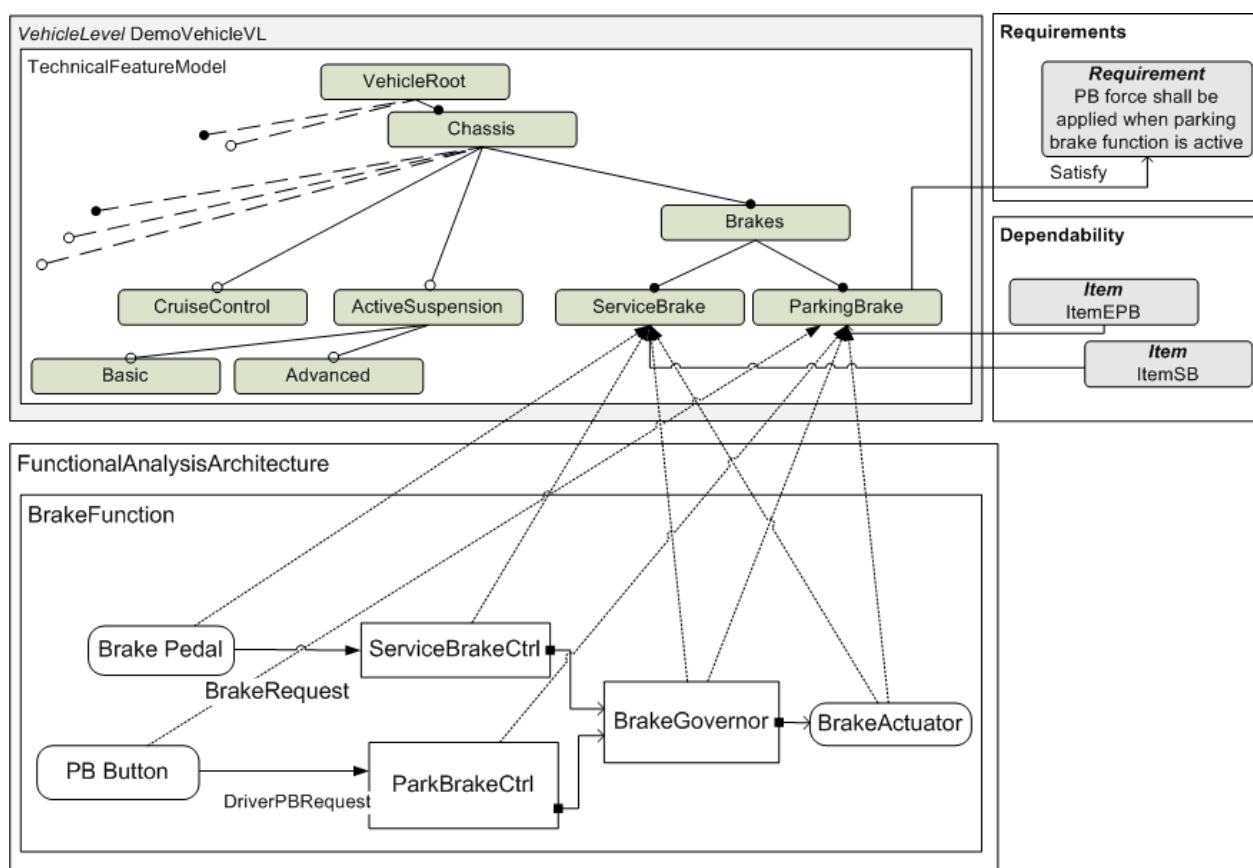
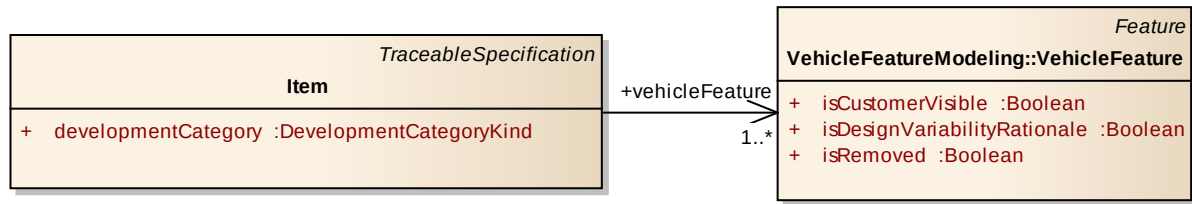


Figure 24. Example of an Item Definition View

### 5.5.2 Related Modeling Concepts



**Figure 25.**  Item and its reference to one or several VehicleFeatures

The Item element references VehicleFeatures, which in turn are referenced by all elements that realize the feature. This means that any EAST-ADL or AUTOSAR element may be included in an item. Figure 26 shows how an identifiable (such as an AUTOSAR SWC or EAST-ADL Function) realizes an EAElement (such as Feature or Function).

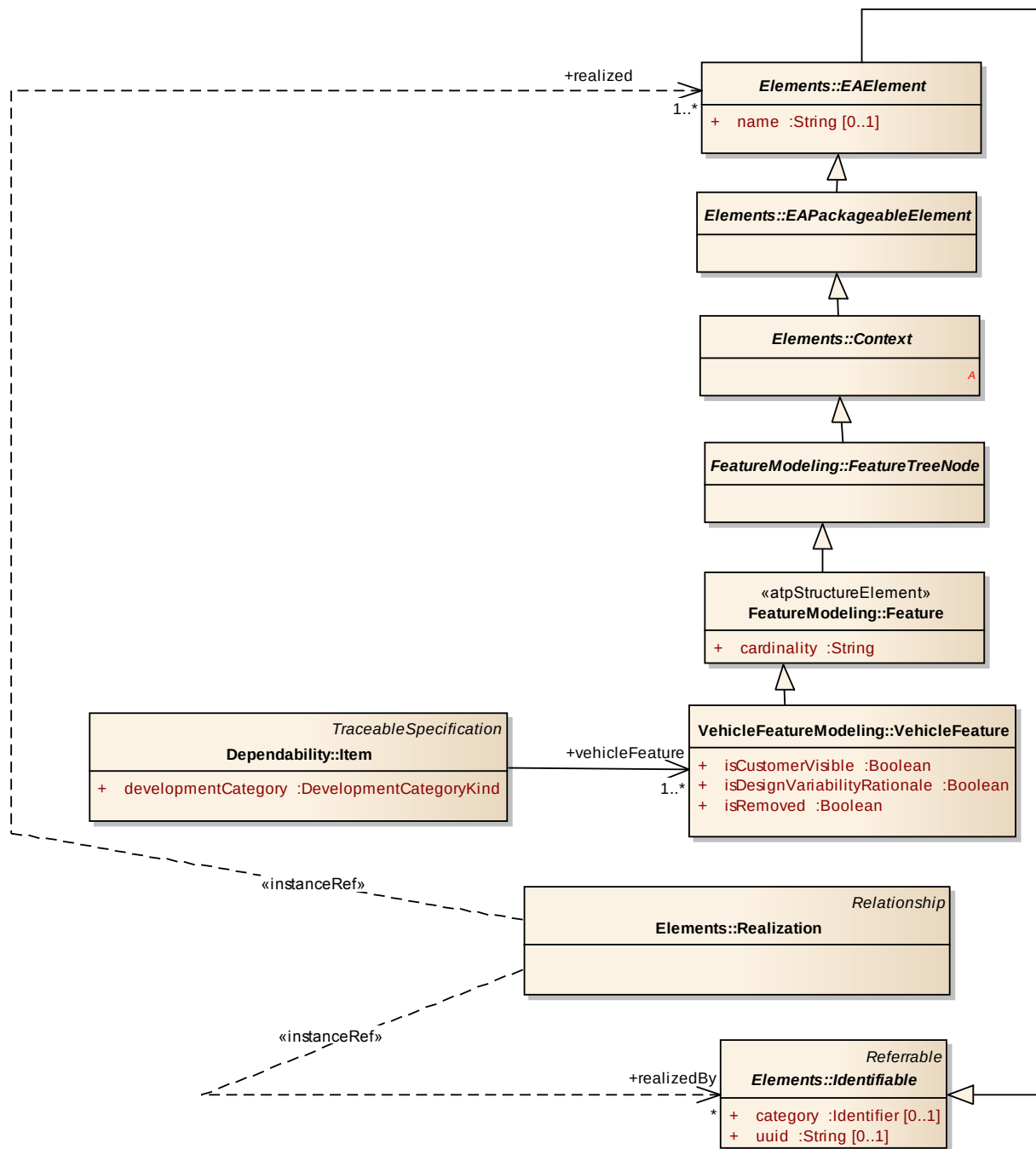


Figure 26.  Realization relation between realized Element and realizing identifiable.

## 5.6 Functional and Technical Safety Concept

Visualizing the Functional Safety Concept or the Technical Safety Concept is important to assess what measures are taken to meet the safety goal. Functional and Technical Safety Concepts are composed of sets of safety requirements allocated to architectural elements.

### 5.6.1 Example View

Figure 27 shows how the functional safety concept may be visualized. In the upper part, its Item elements are also included, although they are not part of the functional safety concept.

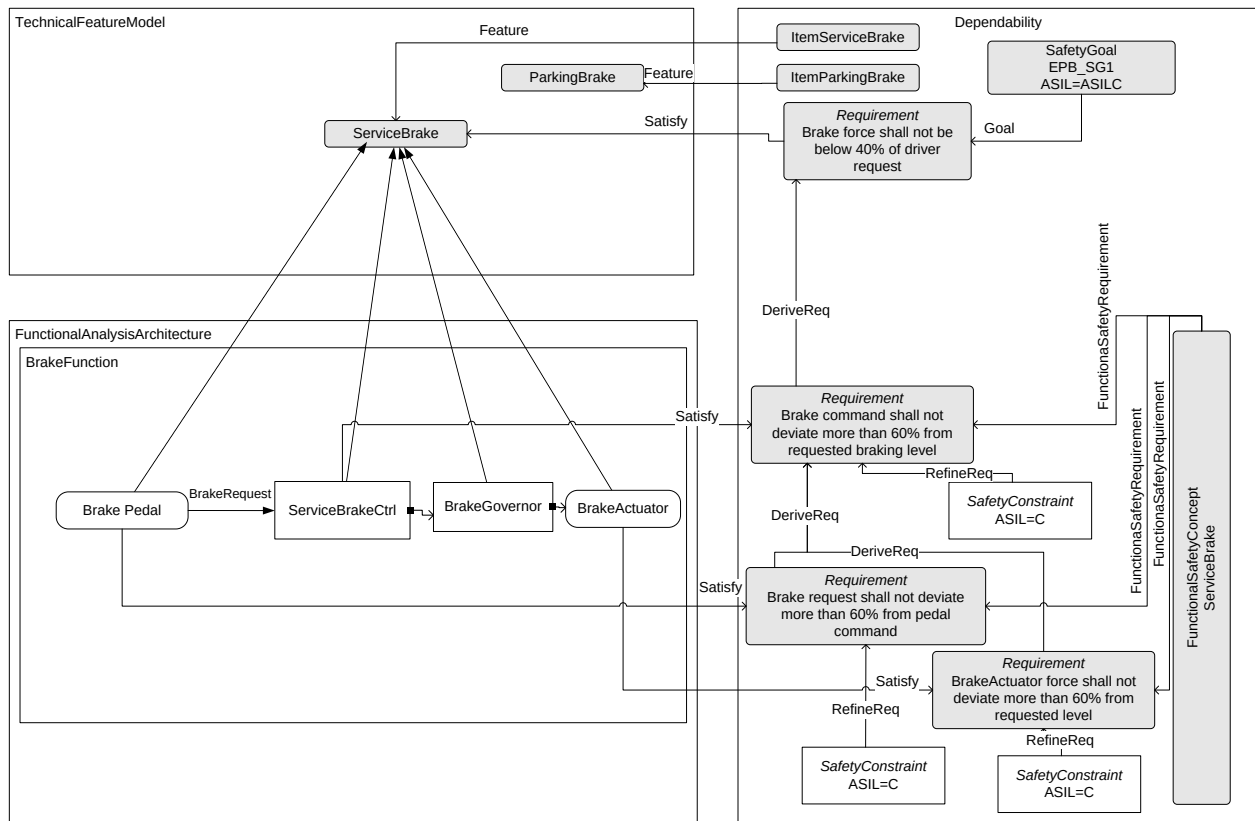


Figure 27.  Functional Safety Concept

## 5.6.2 Modelling Concepts

The views of Functional Safety Concept and the Technical Safety Concept are based on the elements with the same names. Figure 17 above shows the elements and relations.

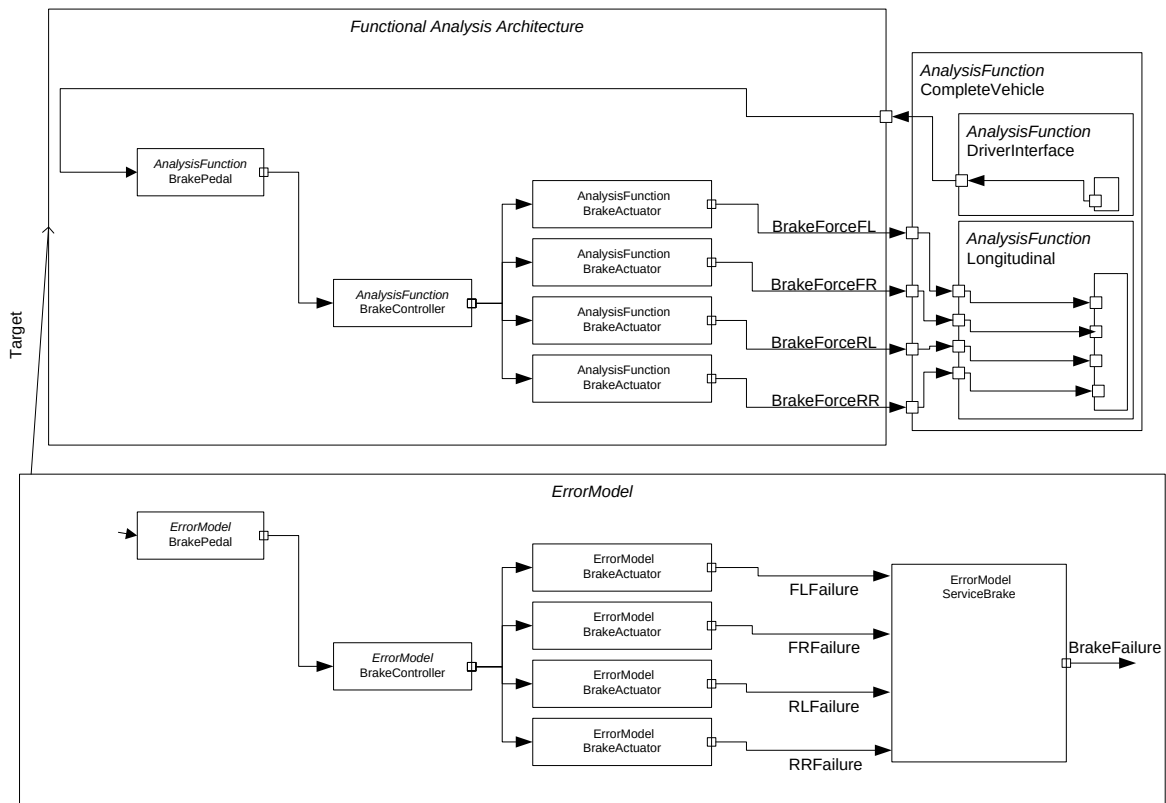
## 5.7 Error Propagation Modelling

Error propagation models have the purpose to define for each component (functional or physical, concrete or abstract) the expected internal faults and expected faults from adjacent components, the error propagation logic and the resulting potential failures of the component.

### 5.7.1 Example View

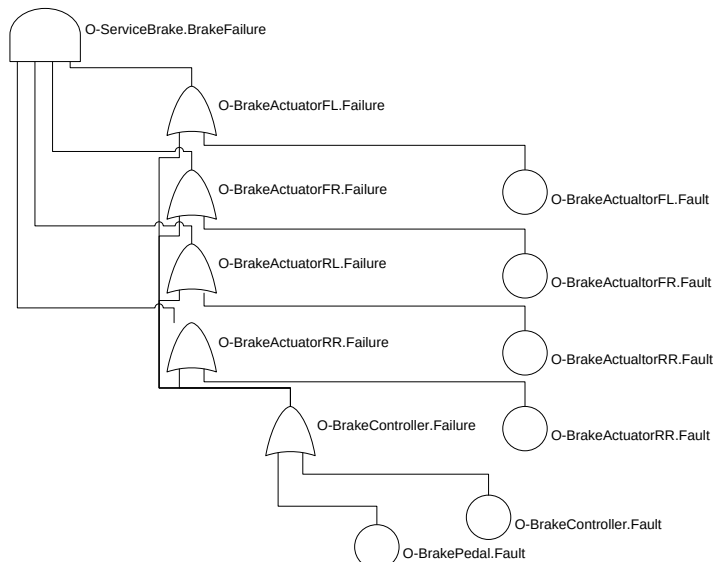
Figure 28 shows a nominal model with one plausible error propagation model. The safety engineer may adjust the error propagation model further, which is why they are separated as two structures.





**Figure 28.** Example of nominal model (top) and its corresponding error propagation model

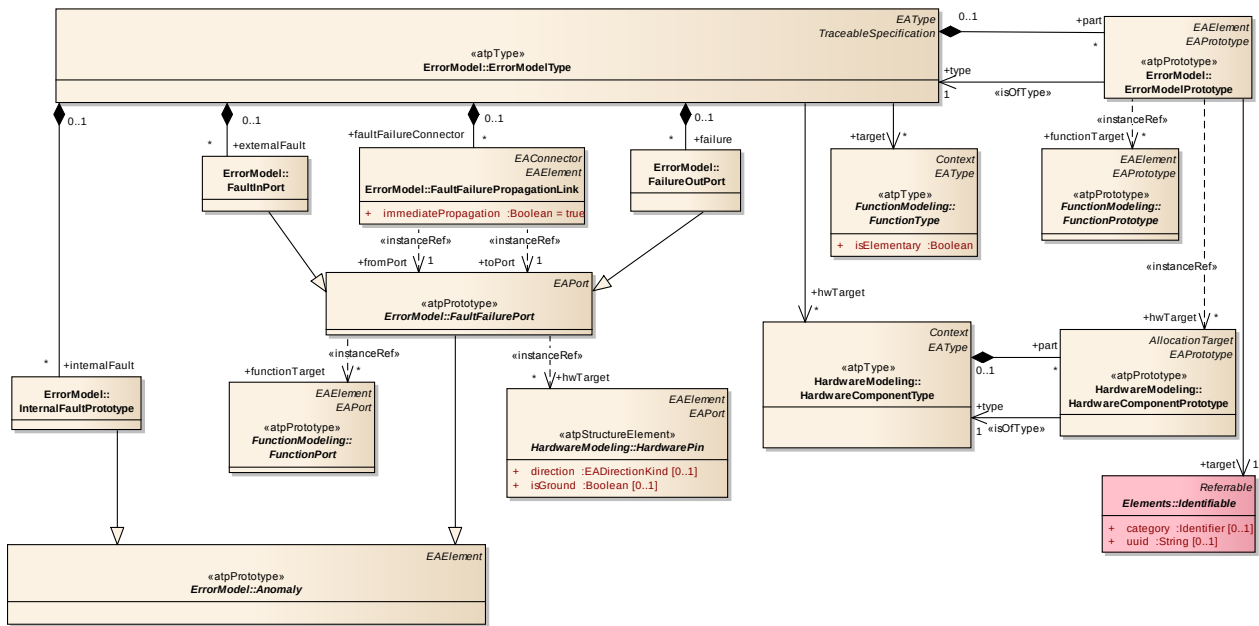
Another view of the error propagation model is to consider the failure logic of each block and derive a Fault Tree, see Figure 29



**Figure 29.** Fault Tree view of the ErrorModel

## 5.7.2 Related Modeling Concepts

Error propagation is modeled using the ErrorModelType hierarchy, as described in Figure 30.



**Figure 30.**  Error propagation modelling is based on the ErrorModelType hierarchy

To derive and show a fault tree, the ErrorBehavior of the error model types must be known. Figure 31 shows the corresponding EAST-ADL elements.

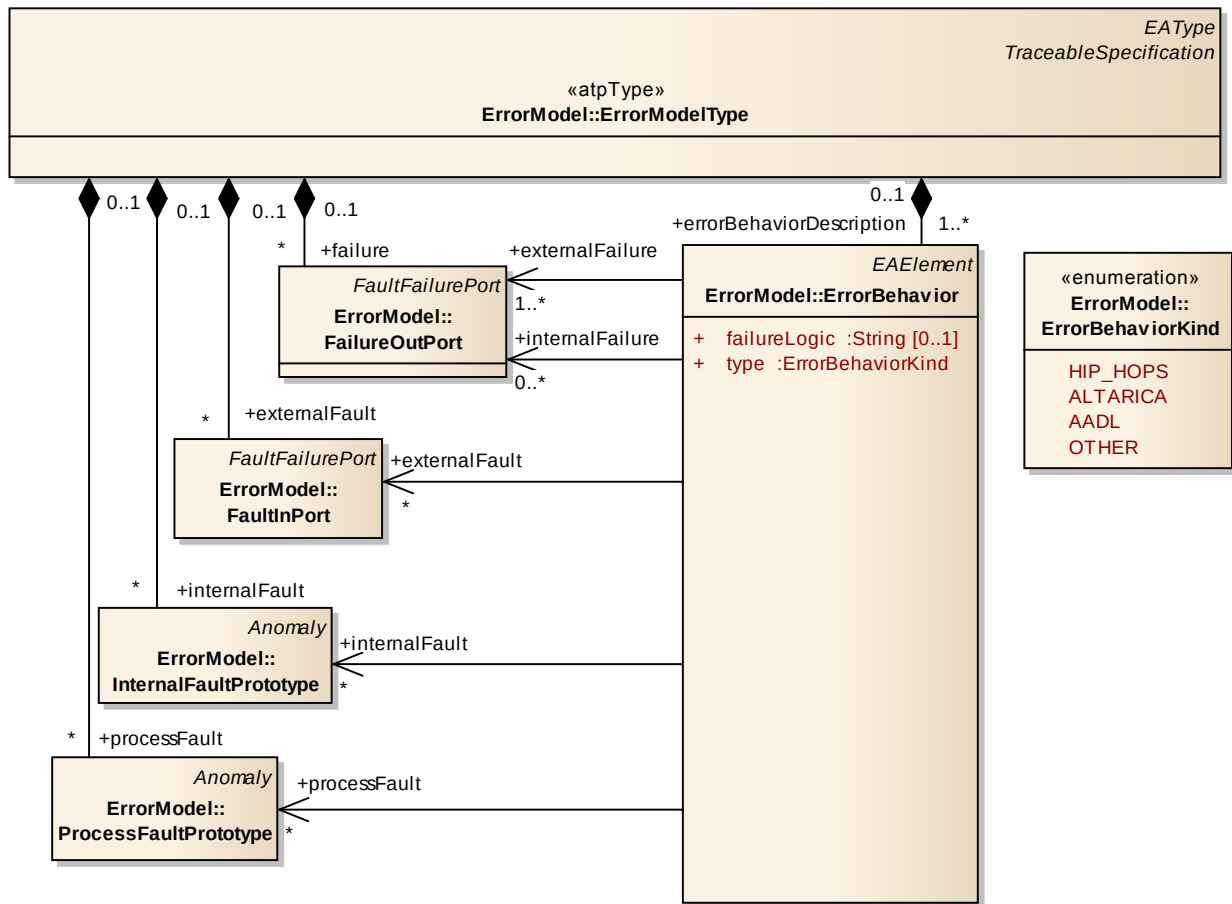


Figure 31.  **ErrorBehavior with failureLogic is the basis for defining error propagations.**

The Error Propagation Model and the related Fault Tree Analysis are two views used to perform analysis which generates artefacts for the Functional Safety Concept and for the Technical Safety Concept.

### 5.7.3 Work-Phases when Eliciting Error Propagation Models

Working with Error Propagation models can be considered in four phases: Defining, Analysing, Understanding and Evolving.

#### 5.7.3.1 Defining

The first step, defining, is to create the error propagation model from the nominal architecture model. Decisions are needed to determine which or what level of components are going to be used as a basis for error propagation model. There are also engineering decisions needed in order to define an efficient error propagation model, for example by identifying the right granularity. One or several Error Model components may capture the error propagation behaviour for a part of the architecture. The kind of errors considered will also be a critical engineering decision. While a lot

of engineering decisions are involved, synthesis and microautomation support from the tooling is important for efficiency.

---

#### 5.7.3.2      Analysing

---

The second step is the analysing process, by means of fault tree analysis [6]. In this step, the fault propagation data in the error model would be synthesized and analysed into a fault tree. It should be noted that fault tree analysis is a wide-spread analysis method with a well-known view. For this reason, the view is not further described here. The analysis tool would automatically generate the corresponding fault tree once the error model is decided.

Different analysis tools and notations (AltaRica, HiP-HOPS, Rodelica, AADL) are available, and the failure logic must be defined accordingly.

---

#### 5.7.3.3      Understanding

---

To understand the generated error propagation model and see its relation to a nominal model, efficient views are needed. This relates both to the error propagation model, and the corresponding fault tree or FMEA table. By visualizing failures, failure propagation paths, etc. it is possible to see how the system elements relates to the identified faults and error propagations.

---

#### 5.7.3.4      Evolving

---

After the initial definition of models, and subsequent analysis, there will be adjustments of the nominal model and of the error propagation model. To secure consistency, modifications of the nominal model should result in the modification of the error model and vice versa.

---

**6 Summary**

---

This chapter summarizes the outcomes of WP2.

The RFQ process for collaboration partners working with the same EAST-ADL-like information model has been defined in Section 3.2.2 followed by a discussion on how requirements and models have to be analyzed. This process forms the context for several metrics and views which have been analyzed and are presented in Chapter 4 and Chapter 5 respectively. Notable information models concern the interaction between manufacturer and supplier. For example, the RFQ-process has its own information modelling considerations, as exchange of models lead to versioning. Consequently, the information modelling must have versioning information. Similarly, extensions using User Defined Attributes make it possible to keep release information in the system model. Other notable information modelling aspects are the task modelling element and the lifecycle of a task.

Tool support to work in accordance with the model-based and information modelling-supported paradigm discussed in Project Synligare, require special considerations that are outlined in Section 3.4 and Section 3.5.

In summary, the information modelling has shown that to use EAST-ADL as information model, it needs to be complemented by at least versioning information, a way to keep three models side by side (from the OEM, from the Tier-1 and a delta) and a way to link EAST-ADL with another information model for keeping information on the visual representation of concrete views.

Chapter 4 lists the metrics that have been defined. They are of two main types, namely progress metrics and product metrics. The progress metrics concern requirements, verification, realization, and safety-related progress. The metrics can be applied in various contexts. Section 4.6 has commented on how to identify the context.

Chapter 5 lists the conceptual views that have resulted from WP2. The conceptual views support various key activities, such as error propagation modelling, getting overview over a distributed system of software components, and working with functional safety.

---

**6.1 Inputs to WP3**

---

The considerations for tooling in Section 3.4 and Section 3.5 list explicit requirements on tooling that is to be based on the considered information modelling.

The metrics in Chapter 4 and the conceptual views in Chapter 5 set the starting point for developing the corresponding algorithms.

---

**6.2 Dissemination**

---

A workshop paper has been accepted to ASCoMS 2014.

---

**7 Bibliography**

---

- [1] AUTOSAR Development Partnership: AUTOSAR Specification. <http://www.autosar.org/>
- [2] AUTOSAR Tool Platform User Group: AUTOSAR Tool platform. <http://www.artop.org/>
- [3] EAST-ADL Association: EAST-ADL 2.1.12 Specification. <http://www.east-adl.info/>
- [4] EAST-ADL Association: EAST-ADL Schema eastadl\_2-1-12.xsd. <http://www.east-adl.info/>
- [5] EATOP Eclipse Open Source Project: EAST-ADL Tool platform. <http://www.eclipse.org/eatop>
- [6] Ericson, Clifton. s.l.: Fault Tree Analysis - A History. free-tree.net, 1999. Proceedings of the International System Safety Conference. pp. 1-9.
- [7] International Organization for Standardization (2011): Road Vehicles – Functional Safety – Part 1 to 9. International Standard ISO/FDIS 26262. November 2011.
- [8] Synligare Consortium: Synligare Project website. <http://www.synligare.eu/>
- [9] Synligare Consortium: Synligare Deliverable D1.1 Needs Identification. <http://www.synligare.eu/>
- [10] Synligare Consortium: Synligare Deliverable D4.1 Validation. <http://www.synligare.eu/>